

MODUL PRAKTIKUM

Prediksi Awal Kesehatan Daun Berbasis Citra Digital

**PROGRAM STUDI
INSTITUSI PENDIDIKAN
2024**

Kata Pengantar

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulisan "Modul Praktikum Prediksi Awal Kesehatan Daun Berbasis Citra Digital" ini dapat diselesaikan dengan baik.

Modul ini dirancang secara khusus untuk memberikan pengalaman praktis secara komprehensif dari awal hingga akhir (end-to-end) dalam membangun sebuah sistem Machine Learning berbasis Computer Vision. Melalui studi kasus klasifikasi kondisi daun menjadi 5 (lima) kelas, pembaca akan dipandu mulai dari tahapan pemuatan data, eksplorasi data, perancangan model jaringan saraf tiruan (ResNet9), pelatihan model, hingga tahapan *deployment* menjadi sebuah aplikasi web sederhana menggunakan Streamlit.

Penggunaan Google Colab sebagai environment utama dalam praktikum ini diharapkan dapat memudahkan pembaca dalam mempraktekkan materi tanpa perlu memikirkan spesifikasi perangkat keras yang tinggi, karena seluruh komputasi dilakukan melalui teknologi *cloud*.

Kami menyadari bahwa modul ini tidak luput dari kekurangan. Oleh karena itu, segala bentuk kritik dan saran yang membangun sangat kami harapkan guna penyempurnaan di masa yang akan datang. Perlu ditegaskan pula bahwa model yang dibangun dalam modul ini berfungsi sebagai purwarupa (prototype) untuk *screening* awal, dan bukan sebagai diagnosis agrikultural yang final.

Akhir kata, semoga modul ini dapat memberikan manfaat yang sebesar-besarnya bagi para pembaca, khususnya bagi pihak yang sedang mendalami implementasi Machine Learning dan Computer Vision di dunia nyata.

Penyusun

Daftar Isi

Kata Pengantar	i
Daftar Gambar	iv
Daftar Tabel	v
Daftar Potongan Kode	vi
Daftar Output Kode	viii
1 Pendahuluan	1
1.1 Gambaran Umum	1
1.2 Tujuan Praktikum	2
1.3 Batasan Sistem	2
1.4 Persiapan Lingkungan Praktikum	2
2 Persiapan dan Eksplorasi Data	4
2.1 Data Loading dengan KaggleHub	4
2.1.1 Pengambilan Data	4
2.1.2 Mengecek Struktur Folder Dataset	5
2.2 Persiapan Environment	6
2.2.1 Mengimpor Library	6
2.2.2 Menginstal Library Tambahan	7
2.3 Exploratory Data Analysis (EDA)	8
2.3.1 Menentukan Lokasi Folder Dataset	8
2.3.2 Menghitung Total Gambar dalam Dataset	9
2.3.3 Menampilkan Contoh Gambar Dataset	10
2.3.4 Identifikasi Kelas Dataset dan Memisahkan Kelas Penyakit dan Kelas Sehat	11
2.3.5 Identifikasi Data yang Akan Digunakan	14
2.3.6 Visualisasi Distribusi Data yang Digunakan	17
2.3.7 Visualisasi Total Gambar per Kelas	18
2.3.8 Ringkasan Jumlah Data Training dan Validation	20
3 Persiapan Data untuk Pemodelan	21
3.1 Menentukan Kelas Target	21
3.2 Membuat Dataset Subset 5 Kelas	22
3.3 Menentukan Transformasi Gambar	23
3.4 Membuat Dataset ImageFolder	24
3.5 Mengecek Bentuk Data Gambar dan Label	25
3.6 Mengecek Jumlah Kelas Dataset	26
3.7 Membuat Fungsi untuk Menampilkan Gambar Dataset	27
3.8 Menentukan Random Seed	27
3.9 Menentukan Batch Size	28
3.10 Membuat DataLoader	29
3.11 Menampilkan Batch Gambar dari DataLoader	30

4	Pemodelan dan Pelatihan	33
4.1	Pemodelan (Modeling)	33
4.1.1	Menyiapkan Device GPU/CPU	33
4.1.2	Memindahkan DataLoader ke Device	35
4.1.3	Membuat Residual Block Sederhana	36
4.1.4	Membuat Fungsi Akurasi dan Base Class Model	37
4.1.5	Membuat Arsitektur Model ResNet9	40
4.1.6	Membuat Object Model ResNet9	43
4.1.7	Menampilkan Ringkasan Arsitektur Model	45
4.2	Pelatihan (Training)	47
4.2.1	Membuat Fungsi Training dan Evaluasi Model	47
4.2.2	Evaluasi Awal Model Sebelum Training	50
4.2.3	Menentukan Hyperparameter Training	51
4.2.4	Menjalankan Training Model	53
5	Visualisasi Hasil dan Deployment	55
5.1	Visualisasi Hasil	55
5.1.1	Menampilkan Ringkasan History Training	55
5.1.2	Visualisasi Grafik Akurasi Model	57
5.1.3	Visualisasi Grafik Loss Model	58
5.1.4	Visualisasi Learning Rate	61
5.1.5	Visualisasi Confusion Matrix	62
5.1.6	Visualisasi Contoh Hasil Prediksi Gambar	65
5.1.7	Classification Report	67
5.2	Deployment	69
5.2.1	Menyimpan Model Hasil Training	70
5.2.2	Menginstall Library Streamlit dan Pyngrok	71
5.2.3	Membuat File Aplikasi Streamlit app.py	72
5.2.4	Menjalankan Aplikasi Streamlit dengan Ngrok	76

Daftar Gambar

1.1	Gambaran Alur Kerja (End-to-End Workflow)	1
1.2	Detail Pipeline Pemrosesan Data dan Pemodelan	1
2.1	Contoh gambar daun dari dataset	11
2.2	Visualisasi Distribusi Jumlah Gambar pada 5 Kelas Target	18
2.3	Visualisasi Total Gambar pada Setiap Kelas Target	19

Daftar Tabel

2.1	Tabel Daftar Tanaman	13
2.2	Tabel Kelas Sehat	13
2.3	Tabel Kelas Penyakit	14
2.4	Tabel Rincian Data Target (Daun Tomat)	17

Daftar Potongan Kode

2.1	Mengunduh dataset dari KaggleHub	4
2.2	Mengecek struktur folder dataset	5
2.3	Mengimpor library yang dibutuhkan	6
2.4	Menginstal library torchsummary	7
2.5	Menentukan lokasi folder dataset	8
2.6	Menghitung total gambar dalam dataset	9
2.7	Menampilkan contoh gambar dataset	10
2.8	Identifikasi dan filter kelas dataset	11
2.9	Menentukan dan menghitung data target	14
2.10	Membuat visualisasi distribusi data target	17
2.11	Membuat visualisasi total gambar	19
2.12	Menghitung ringkasan akhir jumlah data	20
3.1	Menentukan kelas target	21
3.2	Membuat subset dataset 5 kelas tomat	22
3.3	Menentukan transformasi gambar	24
3.4	Membuat dataset dengan ImageFolder	24
3.5	Mengecek bentuk tensor gambar dan label	25
3.6	Mengecek jumlah kelas dataset	26
3.7	Membuat fungsi untuk menampilkan gambar	27
3.8	Menampilkan contoh gambar dataset	27
3.9	Menampilkan contoh gambar lainnya	27
3.10	Menentukan random seed	28
3.11	Menentukan batch size	28
3.12	Membuat DataLoader	29
3.13	Membuat fungsi untuk menampilkan batch gambar	30
3.14	Menampilkan batch gambar training	30
4.1	Fungsi untuk menyiapkan device GPU/CPU	33
4.2	Memindahkan DataLoader ke device	35
4.3	Membuat SimpleResidualBlock	36
4.4	Membuat fungsi akurasi dan base class	38
4.5	Membangun arsitektur ResNet9	40
4.6	Membuat object model ResNet9	43
4.7	Menampilkan ringkasan arsitektur model	45
4.8	Membuat fungsi training dan evaluasi model	47
4.9	Evaluasi awal model sebelum training	51
4.10	Menentukan hyperparameter training	52
4.11	Menjalankan training model	53
5.1	Menampilkan ringkasan history training	55
5.2	Visualisasi grafik akurasi model	57
5.3	Visualisasi grafik loss model	59
5.4	Visualisasi learning rate	61
5.5	Visualisasi confusion matrix	62
5.6	Visualisasi contoh hasil prediksi gambar	65
5.7	Classification report	67
5.8	Menyimpan model hasil training	70
5.9	Menginstall Streamlit dan Pyngrok	71

5.10	Membuat file app.py Streamlit	72
5.11	Menjalankan Streamlit dengan Ngrok	76

Daftar Output Kode

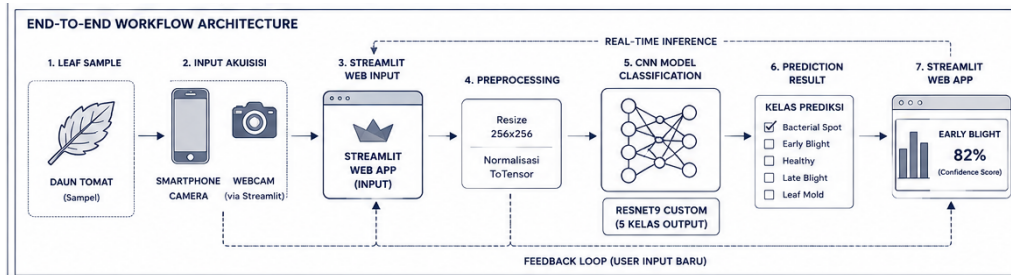
1	Output pengunduhan dataset	5
2	Output struktur folder dataset	5
3	Output impor library	7
4	Output instalasi torchsummary	7
5	Output lokasi folder dataset	9
6	Output total gambar dalam dataset	10
7	Output ringkasan jumlah kelas	13
8	Output ringkasan data target	16
9	Output ringkasan akhir data	20
10	Output penentuan kelas target	22
11	Output pembuatan subset dataset	23
12	Output transformasi gambar	24
13	Output pembuatan dataset ImageFolder	25
14	Output pengecekan bentuk data	26
15	Output jumlah dan daftar kelas	26
16	Output gambar pertama (Bacterial Spot)	27
17	Output gambar Healthy	27
18	Output pengaturan random seed	28
19	Output penentuan batch size	28
20	Output pembuatan DataLoader	29
21	Output batch gambar training	31
22	Output penentuan device komputasi	35
23	Output pemindahan DataLoader ke device	36
24	Output struktur model ResNet9	44
25	Output ringkasan arsitektur model ResNet9	46
26	Output evaluasi awal model	51
27	Output konfigurasi hyperparameter	53
28	Output proses training model	54
29	Output history training (LR list diringkas)	54
30	Output ringkasan history training	56
31	Ilustrasi grafik perkembangan akurasi validation model	58
32	Ilustrasi grafik perkembangan loss model	60
33	Output data learning rate	62
34	Ilustrasi grafik perubahan learning rate	62
35	Ilustrasi confusion matrix model ResNet9	64
36	Ilustrasi contoh hasil prediksi model pada data validation	67
37	Output classification report model ResNet9	69
38	Output penyimpanan model	70
39	Output instalasi Streamlit dan Pyngrok	71
40	Output menjalankan Streamlit dengan Ngrok	78

Bab 1

Pendahuluan

1.1 Gambaran Umum

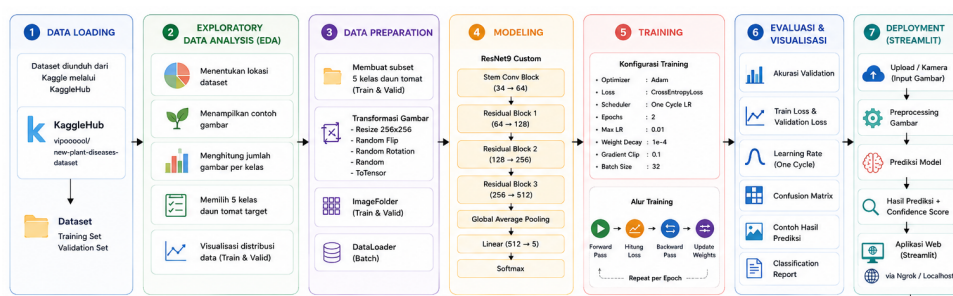
Pada praktikum ini, kita akan membangun sebuah purwarupa (prototype) sistem Machine Learning berbasis Computer Vision yang bertujuan untuk memprediksi secara dini kondisi kesehatan daun tomat. Praktikum ini menggunakan pendekatan end-to-end, mulai dari pemuatan data (*data loading*) hingga fase *deployment* dalam bentuk aplikasi web interaktif.



Gambar 1.1: Gambaran Alur Kerja (End-to-End Workflow)

Sebagaimana diilustrasikan pada Gambar 1.1, alur kerja dalam proyek Machine Learning ini merupakan proses berkesinambungan yang berawal dari pengumpulan dan persiapan dataset daun tomat, diikuti proses perancangan serta pelatihan model jaringan saraf tiruan. Setelah model dievaluasi dan dianggap memadai, sistem diakhiri dengan proses integrasi (deployment) ke dalam platform aplikasi web interaktif agar model siap digunakan oleh para pengguna akhir.

Sistem ini dirancang tidak untuk memberikan diagnosis final layaknya pandangan pakar agrikultur, melainkan digunakan sebagai alat bantu *screening* awal berdasarkan ciri-ciri visual yang tampak pada gambar daun.



Gambar 1.2: Detail Pipeline Pemrosesan Data dan Pemodelan

Lebih rinci lagi, struktur (pipeline) komputasi di bawah kap mesin dapat divisualisasikan pada Gambar 1.2. Alur komputasi ini memperlihatkan transformasi gambar asli ke dalam matriks angka (Tensor) yang sesuai dengan kerangka kerja PyTorch, yang selanjutnya akan diproses secara matematis oleh lapisan-lapisan (layers) pada model arsitektur ResNet9 guna membedakan pola penyakit pada daun tomat.

1.2 Tujuan Praktikum

Adapun tujuan utama dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memberikan pengalaman praktik alur kerja (workflow) pembuatan model Machine Learning secara utuh.
2. Membuat model Machine Learning untuk mengklasifikasikan gambar daun tomat.
3. Menampilkan hasil prediksi model dalam bentuk aplikasi web sederhana.
4. Memahami bahwa model memiliki batasan dan hanya valid untuk data yang sesuai dengan domain pelatihan.

1.3 Batasan Sistem

Untuk memastikan fokus pembelajaran tetap terarah, sistem yang akan dibangun memiliki beberapa batasan:

- Dataset difokuskan secara eksklusif pada **5 kelas kondisi daun tomat**:
 1. Bacterial Spot
 2. Early Blight
 3. Late Blight
 4. Leaf Mold
 5. Healthy (Sehat)
- Sistem tidak dapat digunakan untuk memprediksi jenis tanaman lain.
- Model yang dilatih belum dilengkapi dengan kelas penolakan (seperti kelas 'Unknown' atau 'Non Leaf'), sehingga input gambar diasumsikan selalu berupa daun tomat.

Kelima kelas batasan di atas akan menjadi acuan utama dalam proses seleksi data pada **Bab 2**. Meskipun dataset awal memiliki puluhan kelas dari berbagai jenis tanaman, kita akan menyaringnya hanya untuk mengambil data yang sesuai dengan lima kondisi daun tomat tersebut. Dengan demikian, batasan ini bukan sekadar teori — melainkan akan langsung diimplementasikan pada tahap eksplorasi dan persiapan data.

1.4 Persiapan Lingkungan Praktikum

Pada praktikum ini, spesifikasi komputer lokal yang tinggi tidak diwajibkan karena seluruh proses komputasi, termasuk pelatihan model yang membutuhkan GPU, akan dilakukan di *cloud*. Berikut adalah *tools* dan lingkungan yang perlu dipersiapkan:

Berikut adalah rincian masing-masing teknologi yang menjadi tulang punggung praktikum ini:

- **Google Colab:** Lingkungan Jupyter Notebook gratis berbasis cloud dari Google yang menyediakan akses ke GPU. Digunakan untuk mengeksekusi kode Python, melatih model, dan menjalankan *server* aplikasi. Seluruh kode pada bab-bab selanjutnya akan dijalankan di platform ini.
- **KaggleHub:** Library untuk mengunduh dataset secara langsung dari Kaggle ke environment Google Colab. Akan digunakan pertama kali pada **Bab 2** saat proses pengambilan data.
- **PyTorch dan Torchvision:** Framework Deep Learning yang akan digunakan untuk mengelola dataset dan membangun arsitektur jaringan saraf tiruan (ResNet9). Library ini akan diimpor pada **Bab 2** dan menjadi fondasi utama pemodelan di **Bab 4**.
- **Streamlit:** Framework open-source berbasis Python untuk membuat antarmuka web interaktif khusus Machine Learning. Akan digunakan pada tahap *deployment* setelah model selesai dilatih.
- **Ngrok:** Layanan *tunneling* yang memungkinkan aplikasi Streamlit yang berjalan di *localhost* Google Colab dapat diakses melalui internet menggunakan tautan publik. Juga akan digunakan bersamaan dengan Streamlit pada tahap *deployment*.

Pastikan Anda telah memiliki akun Google untuk menggunakan Google Colab.

Setelah seluruh gambaran dan persiapan lingkungan komputasi telah dipahami, langkah selanjutnya adalah benar-benar mulai bekerja dengan data. Pada **Bab 2**, kita akan memulai proses pengambilan dataset dari KaggleHub, menyiapkan seluruh *library* yang dibutuhkan dalam *environment* Google Colab, serta melakukan eksplorasi data awal (*Exploratory Data Analysis* / EDA) untuk mengenali karakteristik data yang akan menjadi bahan baku pelatihan model.

Bab 2

Persiapan dan Eksplorasi Data

Pada Bab 1, kita telah memahami gambaran umum sistem, tujuan praktikum, serta berbagai *tools* yang akan digunakan seperti Google Colab, PyTorch, Streamlit, dan Ngrok. Sekarang, saatnya untuk mulai bekerja dengan data.

Bab ini akan memandu Anda dalam mempersiapkan seluruh kebutuhan data yang akan digunakan pada praktikum ini. Terdapat tiga tahapan utama yang akan dilalui, yaitu pengambilan data (*data loading*) dari KaggleHub, persiapan *environment* dengan mengimpor seluruh *library* yang dibutuhkan, serta eksplorasi data awal (*Exploratory Data Analysis* / EDA) untuk memahami karakteristik dataset yang akan digunakan.

2.1 Data Loading dengan KaggleHub

Pada Bab 1, kita telah berkenalan dengan KaggleHub sebagai salah satu *library* yang akan digunakan untuk mengunduh dataset dari platform Kaggle. Sekarang, kita akan benar-benar menggunakannya.

Langkah pertama yang harus dilakukan adalah mengambil dataset gambar daun tanaman dari KaggleHub. Dataset akan digunakan sebagai bahan utama untuk melatih model *Machine Learning* agar mampu mengenali kondisi kesehatan tanaman berdasarkan citra daun.

2.1.1 Pengambilan Data

Tahap awal untuk memulai praktikum ini adalah mendatangkan bahan baku berupa ribuan foto tanaman dari repositori *online* ke dalam lingkungan komputasi kita. Dataset yang digunakan berasal dari Kaggle dengan nama `vipooooool/new-plant-diseases-dataset`.

KaggleHub adalah *library* Python yang memungkinkan kita mengunduh dataset dari Kaggle secara langsung tanpa perlu mengunduh secara manual melalui *browser*. Dengan menggunakan KaggleHub, dataset akan secara otomatis diunduh dan diekstrak ke dalam *path* yang dapat langsung digunakan.

Berikut adalah kode untuk mengunduh dataset:

```
1 import kagglehub
2
3 # Mengunduh dataset Plant Disease dari KaggleHub
4 dataset_path = kagglehub.dataset_download(
5     "vipooooool/new-plant-diseases-dataset"
6 )
7
8 # Menampilkan lokasi dataset yang berhasil diunduh
9 print("Dataset berhasil diunduh.")
10 print("Lokasi dataset:", dataset_path)
```

Kode 2.1: Mengunduh dataset dari KaggleHub

Ketika kode di atas dijalankan, KaggleHub akan mengunduh dataset dan menyimpannya ke dalam direktori *cache* lokal. Variabel `dataset_path` akan menyimpan lokasi lengkap dari dataset yang telah diunduh. Lokasi ini nantinya akan digunakan pada tahap selanjutnya untuk mengakses gambar-gambar yang akan diproses.

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Dataset berhasil diunduh.  
2 Lokasi dataset: /kaggle/input/new-plant-diseases-dataset
```

Output 1: Output pengunduhan dataset

Catatan Penting

Pastikan Anda telah terhubung ke internet saat menjalankan kode di atas. Dataset yang diunduh memiliki ukuran yang cukup besar, sehingga proses pengunduhan mungkin memerlukan waktu beberapa saat tergantung pada kecepatan koneksi internet Anda.

2.1.2 Mengecek Struktur Folder Dataset

Setelah dataset berhasil diunduh, langkah berikutnya adalah mengecek isi folder utama dataset. Tahap ini penting karena setiap dataset Kaggle bisa memiliki struktur folder yang berbeda. Dengan melihat isi folder utama, kita dapat menentukan lokasi folder train, valid, dan test yang akan digunakan pada proses pelatihan model.

Berikut adalah kode untuk mengecek struktur folder dataset:

```
1 import os  
2  
3 # Menampilkan lokasi dataset  
4 print("Lokasi dataset:")  
5 print(dataset_path)  
6  
7 # Mengecek isi folder utama dataset  
8 print("\nIsi folder utama dataset:")  
9  
10 for item in os.listdir(dataset_path):  
11     item_path = os.path.join(dataset_path, item)  
12  
13     if os.path.isdir(item_path):  
14         print(f"[Folder] {item}")  
15     else:  
16         print(f"[File] {item}")
```

Kode 2.2: Mengecek struktur folder dataset

Setelah kode di atas dijalankan, akan terlihat struktur folder dataset sebagai berikut:

```
1 Lokasi dataset:  
2 /kaggle/input/new-plant-diseases-dataset  
3  
4 Isi folder utama dataset:  
5 [Folder] New Plant Diseases Dataset(Augmented)  
6 [Folder] new plant diseases dataset(augmented)  
7 [Folder] test
```

Output 2: Output struktur folder dataset

Dari output tersebut, kita dapat melihat bahwa dataset terdiri dari tiga folder utama. Dua di antaranya merupakan folder yang berisi data *training* dan *validation* yang sudah di-*augmentasi*, serta satu folder test yang berisi data pengujian. Pada praktikum ini, kita akan menggunakan folder yang berisi data *augmented* untuk melatih dan mengevaluasi model.

2.2 Persiapan Environment

Setelah dataset berhasil diunduh, langkah selanjutnya adalah menyiapkan *environment* komputasi. Pada Bab 1, kita telah membahas bahwa praktikum ini akan menggunakan PyTorch dan Torchvision sebagai *framework Deep Learning*, serta Matplotlib untuk visualisasi. Sekarang saatnya untuk mengimpor seluruh *library* tersebut ke dalam sesi Google Colab.

Sebelum memulai proses pengolahan data dan pemodelan, kita perlu menyiapkan *environment* dengan mengimpor seluruh *library* yang dibutuhkan. *Library* yang akan digunakan meliputi *library* untuk pengelolaan data, visualisasi, dan *deep learning*.

2.2.1 Mengimpor Library

Berikut adalah kode untuk mengimpor seluruh *library* yang dibutuhkan dalam praktikum ini:

```
1 # Library untuk pengelolaan file dan data
2 import os
3 import numpy as np
4 import pandas as pd
5
6 # Library untuk visualisasi
7 import matplotlib.pyplot as plt
8 from PIL import Image
9
10 # Library utama PyTorch
11 import torch
12 import torch.nn as nn
13 import torch.nn.functional as F
14
15 # Library untuk pengolahan dataset dan DataLoader
16 from torch.utils.data import DataLoader
17 from torchvision.datasets import ImageFolder
18 import torchvision.transforms as transforms
19 from torchvision.utils import make_grid
20
21 # Library untuk melihat ringkasan arsitektur model
22 from torchsummary import summary
23
24 # Menampilkan plot langsung di notebook
25 %matplotlib inline
26
27 print("Semua library berhasil diimport.")
```

Kode 2.3: Mengimpor library yang dibutuhkan

Setelah kode di atas dijalankan, akan muncul output berikut:

```
1 Semua library berhasil diimport.
```

Output 3: Output impor library

Penjelasan masing-masing *library* yang diimpor:

- `os` — Digunakan untuk mengelola file dan direktori.
- `numpy` — Library untuk komputasi numerik dan operasi matriks.
- `pandas` — Library untuk manipulasi dan analisis data tabular.
- `matplotlib.pyplot` — Library untuk membuat grafik dan visualisasi data.
- `PIL.Image` — Library untuk membuka dan memproses gambar.
- `torch` — Library utama PyTorch untuk *deep learning*.
- `torch.nn` — Modul PyTorch untuk membangun arsitektur jaringan saraf tiruan.
- `torch.nn.functional` — Modul PyTorch untuk fungsi-fungsi aktivasi dan *loss*.
- `DataLoader` — Digunakan untuk memuat data dalam bentuk *batch*.
- `ImageFolder` — Dataset loader khusus untuk data gambar terstruktur dalam folder.
- `transforms` — Modul untuk melakukan transformasi gambar.
- `make_grid` — Utility untuk menampilkan kumpulan gambar dalam bentuk grid.
- `torchsummary` — Library untuk menampilkan ringkasan arsitektur model.

2.2.2 Menginstal Library Tambahan

Beberapa *library* mungkin belum tersedia di *environment* Google Colab secara bawaan. Oleh karena itu, kita perlu menginstalnya terlebih dahulu menggunakan `pip`. Salah satu *library* yang perlu diinstal adalah `torchsummary`:

```
1 !pip install torchsummary -q
2
3 print("Library torchsummary berhasil diinstall.")
```

Kode 2.4: Menginstal library `torchsummary`

Output yang akan tampil:

```
1 Library torchsummary berhasil diinstall.
```

Output 4: Output instalasi `torchsummary`

Catatan Penting

Perintah `!pip install` hanya perlu dijalankan satu kali pada awal sesi Google Colab. Jika *runtime* di-reset, Anda perlu menginstal ulang *library* yang tidak termasuk dalam bawaan Colab.

Setelah semua *library* berhasil diimpor dan diinstal, *environment* praktikum siap digunakan untuk tahap selanjutnya, yaitu eksplorasi data awal (*Exploratory Data Analysis*).

2.3 Exploratory Data Analysis (EDA)

Setelah dataset berhasil dimuat dan *environment* siap, tahap berikutnya adalah melakukan *Exploratory Data Analysis* (EDA). EDA digunakan untuk memahami kondisi awal dataset, seperti jumlah total gambar, struktur folder, jumlah kelas, dan sebaran data pada setiap kelas. Tahap ini penting agar kita mengetahui apakah dataset sudah layak digunakan untuk proses *training* model *Machine Learning*.

2.3.1 Menentukan Lokasi Folder Dataset

Sebelum melakukan EDA, kita perlu menentukan lokasi folder dataset yang akan digunakan. Pada dataset ini, data *training* dan *validation* berada di dalam folder New Plant Diseases Dataset(Augmented). Folder *train* digunakan untuk melatih model, sedangkan folder *valid* digunakan untuk mengevaluasi performa model selama proses *training*.

Tahap ini penting karena variabel `train_dir`, `valid_dir`, dan `diseases` akan digunakan pada proses EDA dan *preprocessing* berikutnya.

Berikut adalah kode untuk menentukan lokasi folder dataset:

```
1 import os
2
3 # Path ke folder dataset utama
4 dataset_main = os.path.join(
5     dataset_path,
6     "New Plant Diseases Dataset(Augmented)",
7     "New Plant Diseases Dataset(Augmented)"
8 )
9
10 # Path ke folder training dan validation
11 train_dir = os.path.join(dataset_main, "train")
12 valid_dir = os.path.join(dataset_main, "valid")
13
14 # Menampilkan lokasi folder
15 print("Folder dataset utama :", dataset_main)
16 print("Folder training      :", train_dir)
17 print("Folder validation     :", valid_dir)
18
19 # Mendapatkan daftar kelas (nama folder dalam train)
20 diseases = os.listdir(train_dir)
21 print("Jumlah kelas dataset :", len(diseases))
```

Kode 2.5: Menentukan lokasi folder dataset

Output yang akan tampil:

```
1 Folder dataset utama : /kaggle/input/new-plant-diseases-dataset/New
  Plant
2                               Diseases Dataset(Augmented)/New Plant Diseases
3                               Dataset(Augmented)
4 Folder training       : /kaggle/input/new-plant-diseases-dataset/New
  Plant
5                               Diseases Dataset(Augmented)/New Plant Diseases
6                               Dataset(Augmented)/train
7 Folder validation     : /kaggle/input/new-plant-diseases-dataset/New
  Plant
8                               Diseases Dataset(Augmented)/New Plant Diseases
9                               Dataset(Augmented)/valid
10 Jumlah kelas dataset : 38
```

Output 5: Output lokasi folder dataset

Dari output di atas, kita dapat melihat bahwa dataset utama memiliki total 38 kelas penyakit tanaman. Jumlah ini jauh lebih banyak dari batasan yang telah ditetapkan pada Bab 1, di mana kita hanya akan berfokus pada **5 kelas kondisi daun tomat**. Oleh karena itu, pada subbab selanjutnya kita akan mengidentifikasi dan menyaring kelas-kelas tersebut dari keseluruhan dataset.

2.3.2 Menghitung Total Gambar dalam Dataset

Setelah mengetahui struktur folder, langkah selanjutnya adalah menghitung total seluruh gambar yang tersedia untuk memastikan kita memiliki cukup banyak contoh foto untuk dipelajari oleh model.

Berikut adalah kode untuk menghitung total gambar dalam dataset:

```
1 # Ekstensi file gambar yang akan dihitung
2 image_extensions = (".jpg", ".jpeg", ".png")
3
4 # Variabel untuk menyimpan jumlah total gambar
5 total_images = 0
6
7 # Menelusuri seluruh folder dan subfolder dataset
8 for root, dirs, files in os.walk(dataset_path):
9     image_files = [
10         file for file in files
11         if file.lower().endswith(image_extensions)
12     ]
13
14     total_images += len(image_files)
15
16 # Menampilkan hasil
17 print("Total gambar dalam dataset:", total_images)
```

Kode 2.6: Menghitung total gambar dalam dataset

Setelah kode dijalankan, akan tampil output berikut:

```
1 Total gambar dalam dataset: 175767
```

Output 6: Output total gambar dalam dataset

Kode di atas akan menelusuri seluruh folder dan subfolder dalam dataset, kemudian menghitung jumlah file gambar dengan ekstensi .jpg, .jpeg, dan .png. Hasilnya menampilkan total seluruh gambar yang tersedia di dalam dataset, yaitu sebanyak **175.767** gambar.

Angka ini masih mencakup seluruh kelas dari 14 jenis tanaman yang berbeda. Pada subbab selanjutnya, kita akan menyaringnya dan memfokuskan hanya pada 5 kelas daun tomat yang telah ditetapkan sebagai batasan sistem pada Bab 1.

2.3.3 Menampilkan Contoh Gambar Dataset

Setelah mengetahui jumlah total gambar, langkah selanjutnya adalah melihat contoh gambar dari dataset. Hal ini penting untuk memahami karakteristik visual dari data yang akan diproses, seperti ukuran gambar, kualitas resolusi, dan variasi kondisi daun yang ada.

Berikut adalah kode untuk menampilkan contoh gambar dari dataset:

```
1 # Ekstensi gambar yang akan dicari
2 image_extensions = (".jpg", ".jpeg", ".png")
3
4 # Menyimpan seluruh path gambar
5 image_paths = []
6
7 # Menelusuri seluruh folder dataset
8 for root, dirs, files in os.walk(dataset_path):
9     for file in files:
10         if file.lower().endswith(image_extensions):
11             image_paths.append(os.path.join(root, file))
12
13 # Mengecek apakah ada gambar yang ditemukan
14 if len(image_paths) == 0:
15     print("Tidak ada gambar yang ditemukan. Periksa kembali lokasi dataset.")
16 else:
17     # Mengambil gambar pertama sebagai contoh
18     sample_image_path = image_paths[0]
19
20     # Membuka gambar
21     img = Image.open(sample_image_path).convert("RGB")
22
23     # Menampilkan gambar
24     plt.figure(figsize=(5, 5))
25     plt.imshow(img)
26     plt.axis("off")
27     plt.title("Contoh Gambar Dataset")
28     plt.show()
```

Kode 2.7: Menampilkan contoh gambar dataset

Output yang akan tampil setelah kode dijalankan adalah visualisasi gambar sebagai berikut (lihat Gambar 2.1):



Gambar 2.1: Contoh gambar daun dari dataset

Dari output di atas, kita dapat melihat bahwa setiap gambar dalam dataset memiliki ukuran **256×256** piksel. Gambar contoh yang ditampilkan berasal dari kelas `Tomato___Late_blight` (penyakit hawar daun tomat).

Catatan Penting

Pada lingkungan Google Colab, perintah `plt.show()` akan menampilkan gambar langsung di bawah sel kode. Jika Anda menjalankan kode di *terminal* biasa, pastikan *backend* matplotlib telah dikonfigurasi dengan benar untuk menampilkan grafik.

Informasi ukuran gambar ini akan menjadi acuan penting pada **Bab 3** saat kita menentukan transformasi gambar. Ukuran 256×256 piksel akan kita pertahankan sebagai dimensi input untuk model, sehingga seluruh gambar akan di-*resize* ke ukuran yang sama sebelum dimasukkan ke dalam proses pelatihan.

2.3.4 Identifikasi Kelas Dataset dan Memisahkan Kelas Penyakit dan Kelas Sehat

Selanjutnya, kita akan memetakan variasi data dengan memisahkan seluruh kelas menjadi dua kelompok: tanaman sehat dan berpenyakit. Kelas yang mengandung kata `healthy` akan dikategorikan sebagai tanaman sehat, sedangkan sisanya otomatis masuk ke daftar kelas penyakit. Langkah ini penting untuk mengetahui seberapa kompleks variasi dataset yang kita miliki.

```
1
2 # 1. Memisahkan kelas penyakit dan kelas sehat
3
4 diseases_only = [
5     class_name for class_name in diseases
6     if "healthy" not in class_name.lower()
7 ]
8
9 healthy_classes = [
```

```

10     class_name for class_name in diseases
11     if "healthy" in class_name.lower()
12 ]
13
14 # 2. Mengambil daftar jenis tanaman dari nama kelas
15
16 def get_plant_name(class_name):
17     """
18     Fungsi untuk mengambil nama tanaman dari nama kelas dataset.
19
20     Contoh:
21     Tomato__Early_blight -> Tomato
22     Apple__Black_rot -> Apple
23     """
24     if "___" in class_name:
25         return class_name.split("___")[0]
26     return class_name
27
28 plant_names = sorted(list(set([
29     get_plant_name(class_name) for class_name in diseases
30 ])))
31
32 # 3. Menampilkan ringkasan jumlah kelas
33
34 print("Total semua kelas      :", len(diseases))
35 print("Total kelas penyakit   :", len(diseases_only))
36 print("Total kelas sehat       :", len(healthy_classes))
37 print("Total jenis tanaman     :", len(plant_names))
38
39 # 4. Membuat tabel daftar tanaman
40 import pandas as pd
41
42 df_plants = pd.DataFrame({
43     "No": range(1, len(plant_names) + 1),
44     "Jenis Tanaman": plant_names
45 })
46
47 print("\nTabel Daftar Tanaman:")
48 display(df_plants)
49
50 # 5. Membuat tabel kelas penyakit
51
52 df_only_diseases = pd.DataFrame({
53     "No": range(1, len(diseases_only) + 1),
54     "Kelas Penyakit": diseases_only
55 })
56
57 print("\nTabel Kelas Penyakit:")
58 display(df_only_diseases)
59
60 # 6. Membuat tabel kelas sehat
61
62 df_healthy_classes = pd.DataFrame({
63     "No": range(1, len(healthy_classes) + 1),
64     "Kelas Sehat": healthy_classes
65 })

```

```

66
67 print("\nTabel Kelas Sehat:")
68 display(df_healthy_classes)

```

Kode 2.8: Identifikasi dan filter kelas dataset

Output teks yang akan tampil setelah kode dijalankan:

```

1 Total semua kelas      : 38
2 Total kelas penyakit   : 26
3 Total kelas sehat      : 12
4 Total jenis tanaman    : 14

```

Output 7: Output ringkasan jumlah kelas

Selain output teks di atas, kode tersebut juga akan menampilkan tiga buah tabel secara interaktif. Berikut ini adalah tabel-tabel yang dihasilkan:

Tabel 2.1: Tabel Daftar Tanaman

No	Jenis Tanaman
1	Apple
2	Blueberry
3	Cherry_(including_sour)
4	Corn_(maize)
5	Grape
6	Orange
7	Peach
8	Pepper,_bell
9	Potato
10	Raspberry
11	Soybean
12	Squash
13	Strawberry
14	Tomato

Tabel 2.2: Tabel Kelas Sehat

No	Kelas Sehat
1	Apple___healthy
2	Blueberry___healthy
3	Cherry_(including_sour)___healthy
4	Corn_(maize)___healthy
5	Grape___healthy
6	Peach___healthy
7	Pepper,_bell___healthy
8	Potato___healthy
9	Raspberry___healthy
10	Soybean___healthy
11	Strawberry___healthy
12	Tomato___healthy

Tabel 2.3: Tabel Kelas Penyakit

No	Kelas Penyakit	No	Kelas Penyakit
1	Apple___Apple_scab	14	Potato___Early_blight
2	Apple___Black_rot	15	Potato___Late_blight
3	Apple___Cedar_apple_rust	16	Squash___Powdery_mildew
4	Cherry_(including_sour)___ Powdery_mildew	17	Strawberry___Leaf_scorch
5	Corn_(maize)___ Cercospora_leaf_spot Gray_leaf_...	18	Tomato___Bacterial_spot
6	Corn_(maize)___Common_rust_	19	Tomato___Early_blight
7	Corn_(maize)___ Northern_Leaf_Blight	20	Tomato___Late_blight
8	Grape___Black_rot	21	Tomato___Leaf_Mold
9	Grape___Esca_(Black_Measles)	22	Tomato___Septoria_leaf_spot
10	Grape___ Leaf_blight_(Isariopsis_Leaf_Spot)	23	Tomato___Spider_mites Two- spotted_spider_mite
11	Orange___ Haunglongbing_(Citrus_greening)	24	Tomato___Target_Spot
12	Peach___Bacterial_spot	25	Tomato___ Tomato_Yellow_Leaf_Curl_Virus
13	Pepper_bell___Bacterial_spot	26	Tomato___Tomato_mosaic_virus

Dari tiga tabel di atas, kita dapat melihat bahwa dataset tidak hanya mencakup satu jenis tanaman, melainkan 14 jenis tanaman dengan puluhan kelas penyakit dan kesehatan. Namun, sesuai dengan batasan sistem pada Bab 1, kita hanya akan mengambil data pada tanaman tomat (Tomato) dengan 5 kondisi spesifik. Mari kita lanjut ke subbab berikutnya untuk melakukan seleksi dan menghitung jumlah data target.

2.3.5 Identifikasi Data yang Akan Digunakan

Melihat luasnya cakupan 14 jenis tanaman dan puluhan penyakit dari tabel di atas, melatih AI untuk mengenali semuanya sekaligus tentu membutuhkan komputasi yang sangat besar. Di sinilah batasan sistem yang telah ditetapkan pada Bab 1 menjadi relevan. Seperti yang telah dijelaskan sebelumnya, prototype ini dibatasi pada **5 kelas kondisi daun tomat** saja.

Oleh karena itu, agar prototype sistem lebih fokus dan efisien, ruang lingkup data akan dipersempit khusus pada komoditas daun Tomat — sesuai dengan batasan yang telah direncanakan sejak awal. Secara spesifik, kita hanya akan mengekstrak lima kondisi target: **Bacterial Spot**, **Early Blight**, **Late Blight**, **Leaf Mold**, dan **Healthy**. Mari kita saring dataset ini dan hitung berapa banyak gambar yang siap digunakan untuk melatih model!

```

1
2 # 1. Menentukan kelas yang akan digunakan sesuai proposal
3
4 target_classes = {
5     "Tomato___Bacterial_spot": "Bacterial Spot",
6     "Tomato___Early_blight": "Early Blight",
7     "Tomato___Late_blight": "Late Blight",
8     "Tomato___Leaf_Mold": "Leaf Mold",

```

```

9     "Tomato__healthy": "Healthy"
10 }
11
12 target_class_folders = list(target_classes.keys())
13
14 # 2. Fungsi untuk mencari folder train dan valid secara otomatis
15
16 def find_split_dir(dataset_path, split_name, target_class_folders):
17     """
18     Fungsi untuk mencari folder train atau valid yang berisi kelas
19     target.
20     """
21     for root, dirs, files in os.walk(dataset_path):
22         if os.path.basename(root).lower() == split_name.lower():
23             if any(class_folder in dirs for class_folder in
24 target_class_folders):
25                 return root
26
27     return None
28
29 train_dir = find_split_dir(dataset_path, "train", target_class_folders)
30 valid_dir = find_split_dir(dataset_path, "valid", target_class_folders)
31
32 print("Folder train:", train_dir)
33 print("Folder valid:", valid_dir)
34
35 if train_dir is None:
36     raise FileNotFoundError("Folder train tidak ditemukan atau tidak
37 berisi kelas target.")
38
39 if valid_dir is None:
40     raise FileNotFoundError("Folder valid tidak ditemukan atau tidak
41 berisi kelas target.")
42
43 # 3. Fungsi untuk menghitung jumlah gambar
44
45 image_extensions = (".jpg", ".jpeg", ".png")
46
47 def count_images(folder_path):
48     """
49     Menghitung jumlah file gambar dalam sebuah folder.
50     """
51     total = 0
52
53     if not os.path.exists(folder_path):
54         return 0
55
56     for root, dirs, files in os.walk(folder_path):
57         total += len([
58             file for file in files
59             if file.lower().endswith(image_extensions)
60 ])
61
62     return total
63
64 # 4. Membuat tabel data yang akan digunakan

```



```

61 import pandas as pd
62
63 rows = []
64
65 for original_class, output_label in target_classes.items():
66     train_class_path = os.path.join(train_dir, original_class)
67     valid_class_path = os.path.join(valid_dir, original_class)
68
69     train_count = count_images(train_class_path)
70     valid_count = count_images(valid_class_path)
71
72     plant_name = original_class.split("___")[0]
73     category = "Sehat" if output_label == "Healthy" else "Penyakit"
74
75     rows.append({
76         "Nama Folder Dataset": original_class,
77         "Jenis Tanaman": plant_name,
78         "Output Model": output_label,
79         "Kategori": category,
80         "Jumlah Train": train_count,
81         "Jumlah Valid": valid_count,
82         "Total Gambar": train_count + valid_count
83     })
84
85 df_target_data = pd.DataFrame(rows)
86
87 # 5. Menampilkan ringkasan data yang digunakan
88
89 print("Total kelas yang digunakan :", len(df_target_data))
90 print("Total gambar train           :", df_target_data["Jumlah Train"].
91       sum())
91 print("Total gambar valid           :", df_target_data["Jumlah Valid"].
92       sum())
92 print("Total seluruh gambar         :", df_target_data["Total Gambar"].
93       sum())
93
94 display(df_target_data)

```

Kode 2.9: Menentukan dan menghitung data target

Output teks yang akan tampil setelah kode dijalankan:

```

1 Folder train: /kaggle/input/new-plant-diseases-dataset/New Plant
  Diseases Dataset (Augmented)/New Plant Diseases Dataset (Augmented)/
  train
2 Folder valid: /kaggle/input/new-plant-diseases-dataset/New Plant
  Diseases Dataset (Augmented)/New Plant Diseases Dataset (Augmented)/
  valid
3 Total kelas yang digunakan : 5
4 Total gambar train         : 9281
5 Total gambar valid         : 2319
6 Total seluruh gambar       : 11600

```

Output 8: Output ringkasan data target

Selain output teks di atas, kode tersebut juga akan menghasilkan tabel interaktif yang berisi rincian jumlah gambar untuk masing-masing kelas target. Berikut adalah representasi tabel tersebut:

Tabel 2.4: Tabel Rincian Data Target (Daun Tomat)

Nama Folder Dataset	Jenis Tanaman	Output Model	Kategori	Jml Train	Jml Valid	Total
Tomato___Bacterial_spot	Tomato	Bacterial Spot	Penyakit	1702	425	2127
Tomato___Early_blight	Tomato	Early Blight	Penyakit	1920	480	2400
Tomato___Late_blight	Tomato	Late Blight	Penyakit	1851	463	2314
Tomato___Leaf_Mold	Tomato	Leaf Mold	Penyakit	1882	470	2352
Tomato___healthy	Tomato	Healthy	Sehat	1926	481	2407

2.3.6 Visualisasi Distribusi Data yang Digunakan

Setelah lima kelas target ditentukan, langkah selanjutnya adalah menampilkan distribusi jumlah gambar pada masing-masing kelas. Visualisasi ini berguna untuk melihat perbandingan proporsi data *training* dan *validation* pada kelas **Bacterial Spot**, **Early Blight**, **Late Blight**, **Leaf Mold**, dan **Healthy**. Pemeriksaan ini penting karena ketidakseimbangan jumlah data antar kelas dapat memengaruhi performa model saat proses pelatihan.

```

1
2 # 1. Validasi dataframe
3
4 required_columns = [
5     "Output Model",
6     "Jumlah Train",
7     "Jumlah Valid",
8     "Total Gambar"
9 ]
10
11 for column in required_columns:
12     if column not in df_target_data.columns:
13         raise ValueError(f"Kolom '{column}' tidak ditemukan pada
14                             df_target_data.")
15
16 # 2. Membuat dataframe untuk visualisasi
17
18 plot_data = df_target_data[
19     ["Output Model", "Jumlah Train", "Jumlah Valid"]
20 ].set_index("Output Model")
21
22 # 3. Membuat bar chart jumlah train dan valid
23
24 plt.figure(figsize=(12, 6))
25
26 ax = plot_data.plot(
27     kind="bar",
28     figsize=(12, 6)
29 )
30
31 plt.title("Distribusi Jumlah Gambar pada Data yang Digunakan")

```

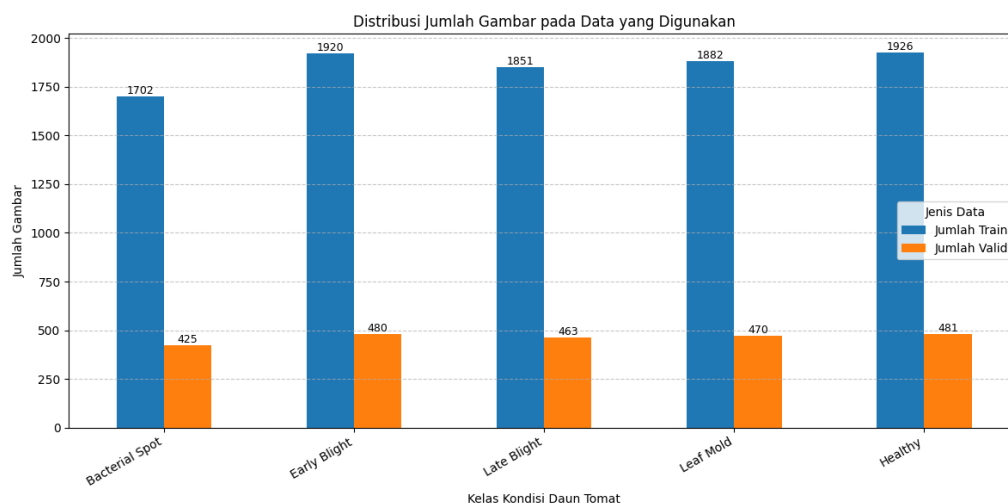
```

31 plt.xlabel("Kelas Kondisi Daun Tomat")
32 plt.ylabel("Jumlah Gambar")
33 plt.xticks(rotation=30, ha="right")
34 plt.grid(axis="y", linestyle="--", alpha=0.7)
35 plt.legend(title="Jenis Data")
36
37 # Menampilkan angka di atas setiap bar
38 for container in ax.containers:
39     ax.bar_label(container, fontsize=9)
40
41 plt.tight_layout()
42 plt.show()

```

Kode 2.10: Membuat visualisasi distribusi data target

Output grafik dari kode di atas adalah sebagai berikut:



Gambar 2.2: Visualisasi Distribusi Jumlah Gambar pada 5 Kelas Target

Dari grafik di atas, terlihat bahwa kelima kelas target memiliki distribusi jumlah gambar yang relatif seimbang, yakni berkisar antara 1.700 hingga 1.900 gambar untuk data *training*, dan 420 hingga 480 gambar untuk data *validation*. Keseimbangan proporsi ini merupakan indikator yang sangat baik. Dengan jumlah sampel yang setara, model AI dapat mempelajari karakteristik setiap kelas secara merata dan mencegah terjadinya bias prediksi ke salah satu kondisi daun tertentu.

Setelah melihat perbandingan data *training* dan *validation*, selanjutnya kita akan melihat total akumulasi gambar pada setiap kelas untuk memastikan tidak ada kelas yang kekurangan sampel secara keseluruhan.

2.3.7 Visualisasi Total Gambar per Kelas

Selain membandingkan porsi data *training* dan *validation*, kita juga perlu melihat akumulasi total gambar secara keseluruhan pada setiap kelas. Langkah ini merupakan penegasan akhir untuk memastikan bahwa tidak ada kelas yang mendominasi ataupun kekurangan sampel sebelum kita melangkah ke tahapan prapemrosesan dan pelatihan model.

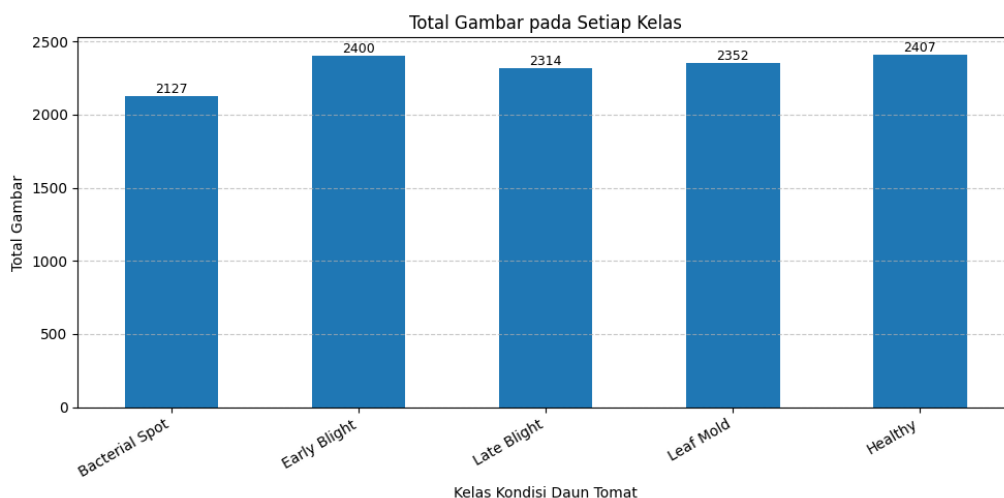
```

1 plt.figure(figsize=(10, 5))
2
3
4 ax = df_target_data.plot(
5     x="Output Model",
6     y="Total Gambar",
7     kind="bar",
8     legend=False,
9     figsize=(10, 5)
10 )
11
12 plt.title("Total Gambar pada Setiap Kelas")
13 plt.xlabel("Kelas Kondisi Daun Tomat")
14 plt.ylabel("Total Gambar")
15 plt.xticks(rotation=30, ha="right")
16 plt.grid(axis="y", linestyle="--", alpha=0.7)
17
18 # Menampilkan angka di atas bar
19 for container in ax.containers:
20     ax.bar_label(container, fontsize=9)
21
22 plt.tight_layout()
23 plt.show()

```

Kode 2.11: Membuat visualisasi total gambar

Output grafik total gambar adalah sebagai berikut:



Gambar 2.3: Visualisasi Total Gambar pada Setiap Kelas Target

Berdasarkan grafik di atas, kelima kelas target secara konsisten memiliki akumulasi total antara 2.100 hingga 2.400 gambar. Angka yang proporsional ini memastikan bahwa dataset kita sudah sepenuhnya ideal dan siap untuk diproses lebih lanjut ke tahap pemodelan.

Sebagai langkah terakhir dari rangkaian EDA, mari kita hitung angka final keseluruhan data *training* dan *validation* yang akan menjadi bahan baku pelatihan model.

2.3.8 Ringkasan Jumlah Data Training dan Validation

Sebagai penutup dari tahapan eksplorasi data, kita akan merangkum kembali angka final keseluruhan gambar yang telah disaring. Ringkasan ini memastikan kepastian total data *training* dan *validation* yang akan dilibatkan secara langsung ke dalam proses pelatihan dan evaluasi model.

```
1
2 # Menghitung total gambar pada data training
3 n_train = df_target_data["Jumlah Train"].sum()
4
5 # Menghitung total gambar pada data validation
6 n_valid = df_target_data["Jumlah Valid"].sum()
7
8 # Menghitung total seluruh gambar yang digunakan
9 n_total = df_target_data["Total Gambar"].sum()
10
11 # Menampilkan ringkasan jumlah data
12 print("Jumlah gambar untuk training   :", n_train)
13 print("Jumlah gambar untuk validation :", n_valid)
14 print("Total gambar yang digunakan   :", n_total)
```

Kode 2.12: Menghitung ringkasan akhir jumlah data

Output dari perhitungan di atas adalah sebagai berikut:

```
1 Jumlah gambar untuk training   : 9281
2 Jumlah gambar untuk validation : 2319
3 Total gambar yang digunakan   : 11600
```

Output 9: Output ringkasan akhir data

Setelah melalui seluruh rangkaian EDA, kita kini memiliki pemahaman yang jelas tentang dataset yang akan digunakan. Dari total 38 kelas dan 175.767 gambar dalam dataset awal, kita telah mempersempit fokus pada **5 kelas kondisi daun tomat** dengan total **11.600 gambar** yang terdiri dari 9.281 data *training* dan 2.319 data *validation*. Distribusi data yang relatif seimbang antar kelas juga telah terkonfirmasi melalui visualisasi, sehingga dataset siap untuk diproses ke tahap berikutnya.

Pada **Bab 3**, kita akan menyiapkan data ini agar benar-benar siap untuk dimasukkan ke dalam model. Tahapan tersebut meliputi pembuatan subset dataset khusus 5 kelas, penerapan transformasi gambar, pembuatan ImageFolder dan DataLoader, serta verifikasi visual bahwa data telah termuat dengan benar.

Bab 3

Persiapan Data untuk Pemodelan

Pada Bab 2, kita telah berhasil mengunduh dataset dari KaggleHub, menyiapkan *environment*, dan melakukan eksplorasi data awal. Melalui EDA, kita memperoleh gambaran bahwa dari total 38 kelas tanaman, prototype ini akan berfokus pada **5 kelas kondisi daun tomat** — yaitu **Bacterial Spot**, **Early Blight**, **Late Blight**, **Leaf Mold**, dan **Healthy** — dengan total 11.600 gambar yang telah terkonfirmasi distribusinya relatif seimbang.

Kini setelah data terpilih dan karakteristiknya kita pahami, langkah berikutnya adalah menyiapkan data tersebut agar benar-benar siap digunakan untuk proses *training* model.

Pada fase ini, dataset yang telah difilter akan melalui serangkaian tahapan teknis: pembuatan subset dataset khusus, penerapan transformasi gambar, pembacaan data menggunakan ImageFolder, hingga pembuatan DataLoader yang akan menjadi jembatan antara data mentah dengan proses pelatihan model pada bab selanjutnya.

3.1 Menentukan Kelas Target

Pada Bab 2, kita telah berhasil mengidentifikasi 5 kelas target dari total 38 kelas yang tersedia dalam dataset. Proses identifikasi dan pemfilteran tersebut telah menghasilkan daftar kelas yang sesuai dengan batasan sistem yang ditetapkan pada Bab 1.

Sekarang, kita akan mendefinisikan secara eksplisit kelima kelas tersebut dalam bentuk dictionary Python. Dictionary ini akan memetakan nama folder asli dari dataset Kaggle (seperti `Tomato___Bacterial_spot`) menjadi nama kelas yang lebih rapi dan mudah dibaca (seperti `Bacterial Spot`). Pemetaan ini penting karena akan digunakan berulang kali pada tahap-tahap selanjutnya, mulai dari pembuatan subset dataset hingga pelatihan model.

```
1 # FASE 3.1 - Menentukan Kelas Target
2
3 target_classes = {
4     "Tomato___Bacterial_spot": "Bacterial Spot",
5     "Tomato___Early_blight": "Early Blight",
6     "Tomato___Late_blight": "Late Blight",
7     "Tomato___Leaf_Mold": "Leaf Mold",
8     "Tomato___healthy": "Healthy"
9 }
10
11 print("Kelas target yang akan digunakan:")
12
13 for original_class, output_label in target_classes.items():
14     print(f"- {original_class} -> {output_label}")
```

Kode 3.1: Menentukan kelas target

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Kelas target yang akan digunakan:
2 - Tomato___Bacterial_spot -> Bacterial Spot
3 - Tomato___Early_blight -> Early Blight
4 - Tomato___Late_blight -> Late Blight
5 - Tomato___Leaf_Mold -> Leaf Mold
6 - Tomato___healthy -> Healthy

```

Output 10: Output penentuan kelas target

3.2 Membuat Dataset Subset 5 Kelas

Setelah kelas target didefinisikan, langkah berikutnya adalah mengekstrak data yang sesuai dari dataset asli. Pada Bab 2, kita telah mengetahui bahwa total gambar yang akan digunakan adalah **11.600 gambar**, terdiri dari 9.281 data *training* dan 2.319 data *validation*. Namun, data tersebut masih berada di dalam folder dataset asli yang bercampur dengan kelas-kelas tanaman lain.

Agar model hanya belajar dari kelas yang sesuai dengan prototype, kita perlu membuat folder dataset baru yang hanya berisi kelima kelas target. Folder baru ini akan memisahkan data daun tomat dari tanaman lainnya, sehingga memudahkan proses pembahasan data pada tahap selanjutnya.

Folder baru ini akan berisi data training dan validation dengan nama kelas yang sudah dirapikan (Bacterial Spot, Early Blight, dll.), bukan lagi nama folder asli dari Kaggle (Tomato___Bacterial_spot, dll.).

```

1 import os
2 import shutil
3
4 # Folder sumber dataset
5 data_dir = os.path.join(
6     dataset_path,
7     "New Plant Diseases Dataset(Augmented)",
8     "New Plant Diseases Dataset(Augmented)"
9 )
10
11 train_dir = os.path.join(data_dir, "train")
12 valid_dir = os.path.join(data_dir, "valid")
13
14 # Folder tujuan subset dataset
15 subset_dir = "/content/tomato_5class_dataset"
16 subset_train_dir = os.path.join(subset_dir, "train")
17 subset_valid_dir = os.path.join(subset_dir, "valid")
18
19 # Membuat folder utama subset
20 os.makedirs(subset_train_dir, exist_ok=True)
21 os.makedirs(subset_valid_dir, exist_ok=True)
22
23 # Menyalin data target dari dataset asli ke folder subset
24 for original_class, output_label in target_classes.items():
25     source_train_path = os.path.join(train_dir, original_class)
26     source_valid_path = os.path.join(valid_dir, original_class)
27
28     target_train_path = os.path.join(subset_train_dir, output_label)

```

```

29     target_valid_path = os.path.join(subset_valid_dir, output_label)
30
31     if os.path.exists(source_train_path):
32         shutil.copytree(source_train_path, target_train_path,
33             dirs_exist_ok=True)
34         print(f"Train berhasil disiapkan: {original_class} -> {
35             output_label}")
36     else:
37         print(f"Folder train tidak ditemukan: {source_train_path}")
38
39     if os.path.exists(source_valid_path):
40         shutil.copytree(source_valid_path, target_valid_path,
41             dirs_exist_ok=True)
42         print(f"Valid berhasil disiapkan: {original_class} -> {
43             output_label}")
44     else:
45         print(f"Folder valid tidak ditemukan: {source_valid_path}")
46
47 print("\nSubset dataset selesai dibuat.")
48 print("Lokasi subset dataset:", subset_dir)

```

Kode 3.2: Membuat subset dataset 5 kelas tomat

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Train berhasil disiapkan: Tomato___Bacterial_spot -> Bacterial Spot
2 Valid berhasil disiapkan: Tomato___Bacterial_spot -> Bacterial Spot
3 Train berhasil disiapkan: Tomato___Early_blight -> Early Blight
4 Valid berhasil disiapkan: Tomato___Early_blight -> Early Blight
5 Train berhasil disiapkan: Tomato___Late_blight -> Late Blight
6 Valid berhasil disiapkan: Tomato___Late_blight -> Late Blight
7 Train berhasil disiapkan: Tomato___Leaf_Mold -> Leaf Mold
8 Valid berhasil disiapkan: Tomato___Leaf_Mold -> Leaf Mold
9 Train berhasil disiapkan: Tomato___healthy -> Healthy
10 Valid berhasil disiapkan: Tomato___healthy -> Healthy
11
12 Subset dataset selesai dibuat.
13 Lokasi subset dataset: /content/tomato_5class_dataset

```

Output 11: Output pembuatan subset dataset

3.3 Menentukan Transformasi Gambar

Sebelum gambar dimasukkan ke model, gambar perlu diproses terlebih dahulu. Pada Bab 2, kita telah melihat contoh gambar dataset dan mengetahui bahwa setiap gambar dalam dataset ini sudah memiliki ukuran **256×256 piksel**. Ukuran ini akan kita pertahankan sebagai ukuran input model.

Selain mengubah ukuran, kita juga perlu mengonversi gambar — yang awalnya berupa file `.jpg` atau `.png` — menjadi format tensor agar dapat dibaca oleh PyTorch. Untuk data training, kita juga menambahkan augmentasi gambar (*random horizontal flip* dan *random rotation*) guna memperkaya variasi data dan mencegah model *overfitting*. Sementara untuk data validation, cukup dilakukan *resize* dan konversi ke tensor tanpa augmentasi.


```

1 IMAGE_SIZE = 256
2
3 train_transform = transforms.Compose([
4     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
5     transforms.RandomHorizontalFlip(),
6     transforms.RandomRotation(10),
7     transforms.ToTensor()
8 ])
9
10 valid_transform = transforms.Compose([
11     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
12     transforms.ToTensor()
13 ])
14
15 print("Transformasi gambar berhasil disiapkan.")

```

Kode 3.3: Menentukan transformasi gambar

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Transformasi gambar berhasil disiapkan.

```

Output 12: Output transformasi gambar

3.4 Membuat Dataset ImageFolder

Setelah subset dataset lima kelas berhasil dibuat dan transformasi gambar ditentukan, langkah berikutnya adalah membaca dataset menggunakan ImageFolder. Pada tahap ini, kita akan memverifikasi apakah jumlah data yang terbaca sesuai dengan temuan pada Bab 2 — yaitu 9.281 data *training* dan 2.319 data *validation*.

ImageFolder akan membaca data berdasarkan struktur folder. Setiap subfolder dianggap sebagai satu kelas. Pada tahap ini, dataset yang dibaca adalah dataset subset yang hanya berisi lima kelas kondisi daun tomat, yaitu Bacterial Spot, Early Blight, Late Blight, Leaf Mold, dan Healthy.

```

1 from torchvision.datasets import ImageFolder
2 import torchvision.transforms as transforms
3
4 # Ukuran gambar yang digunakan sebagai input model
5 IMAGE_SIZE = 256
6
7 # Transformasi untuk data training
8 train_transform = transforms.Compose([
9     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
10    transforms.RandomHorizontalFlip(),
11    transforms.RandomRotation(10),
12    transforms.ToTensor()
13 ])
14
15 # Transformasi untuk data validation
16 valid_transform = transforms.Compose([
17     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
18     transforms.ToTensor()

```

```

19 ])
20
21 # Membaca dataset subset 5 kelas menggunakan ImageFolder
22 train = ImageFolder(subset_train_dir, transform=train_transform)
23 valid = ImageFolder(subset_valid_dir, transform=valid_transform)
24
25 # Mengambil daftar kelas
26 classes = train.classes
27 num_classes = len(classes)
28
29 # Menampilkan informasi dataset
30 print("Dataset training berhasil dibuat.")
31 print("Dataset validation berhasil dibuat.")
32 print("Jumlah kelas:", num_classes)
33 print("Daftar kelas:", classes)
34 print("Jumlah data training:", len(train))
35 print("Jumlah data validation:", len(valid))
36
37 # Validasi agar kelas train dan valid sama
38 if train.classes != valid.classes:
39     raise ValueError("Kelas pada data training dan validation tidak sama.")
40
41 print("Kelas training dan validation sudah sesuai.")

```

Kode 3.4: Membuat dataset dengan ImageFolder

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Dataset training berhasil dibuat.
2 Dataset validation berhasil dibuat.
3 Jumlah kelas: 5
4 Daftar kelas: ['Bacterial Spot', 'Early Blight', 'Healthy', 'Late
   Blight', 'Leaf Mold']
5 Jumlah data training: 9281
6 Jumlah data validation: 2319
7 Kelas training dan validation sudah sesuai.

```

Output 13: Output pembuatan dataset ImageFolder

3.5 Mengecek Bentuk Data Gambar dan Label

Setelah dataset berhasil dibaca menggunakan ImageFolder, langkah berikutnya adalah mengecek bentuk data gambar dan label. Pada Bab 2, kita telah melihat bahwa setiap gambar dalam dataset asli berukuran 256×256 piksel. Sekarang, setelah melalui transformasi, kita dapat memverifikasi bahwa bentuk tensor gambar yang dihasilkan adalah [3, 256, 256] — yaitu 3 *channel* warna (RGB) dengan tinggi dan lebar 256 piksel.

Pada tahap ini, kita mengambil satu contoh data dari dataset training. Gambar akan ditampilkan dalam bentuk tensor, sedangkan label akan ditampilkan dalam bentuk angka sesuai urutan kelas.

```

1 # Mengambil satu contoh gambar dan label dari dataset training
2 img, label = train[0]

```

```

3
4 # Menampilkan bentuk tensor gambar dan label numerik
5 print("Shape gambar :", img.shape)
6 print("Label angka :", label)
7 print("Nama kelas :", classes[label])

```

Kode 3.5: Mengecek bentuk tensor gambar dan label

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Shape gambar : torch.Size([3, 256, 256])
2 Label angka  : 0
3 Nama kelas   : Bacterial Spot

```

Output 14: Output pengecekan bentuk data

3.6 Mengecek Jumlah Kelas Dataset

Setelah dataset berhasil dimuat menggunakan ImageFolder, kita perlu mengecek jumlah kelas yang tersedia. Pada Bab 2, kita telah mengidentifikasi bahwa prototype ini akan menggunakan **5 kelas kondisi daun tomat**. Sekarang, kita akan memverifikasi apakah ImageFolder membaca jumlah kelas yang sesuai dengan identifikasi tersebut.

Tahap ini penting karena jumlah kelas dataset harus sesuai dengan output model. Pada prototype ini, jumlah kelas yang diharapkan adalah **5 kelas**, yaitu Bacterial Spot, Early Blight, Healthy, Late Blight, dan Leaf Mold — sebagaimana telah ditetapkan pada batasan sistem di Bab 1 dan dikonfirmasi melalui EDA di Bab 2.

```

1 # Menghitung jumlah kelas pada dataset training
2 num_classes = len(train.classes)
3
4 # Menampilkan jumlah kelas dan daftar kelas
5 print("Jumlah kelas:", num_classes)
6 print("Daftar kelas:")
7
8 for index, class_name in enumerate(train.classes):
9     print(f"{index}: {class_name}")

```

Kode 3.6: Mengecek jumlah kelas dataset

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Jumlah kelas: 5
2 Daftar kelas:
3 0: Bacterial Spot
4 1: Early Blight
5 2: Healthy
6 3: Late Blight
7 4: Leaf Mold

```

Output 15: Output jumlah dan daftar kelas

3.7 Membuat Fungsi untuk Menampilkan Gambar Dataset

Setelah gambar dan label berhasil dibaca, langkah berikutnya adalah membuat fungsi untuk menampilkan gambar. Pada Bab 2, kita telah melihat contoh gambar dataset menggunakan `plt.imshow()` secara langsung. Sekarang, kita akan membuat fungsi khusus yang lebih praktis untuk menampilkan gambar beserta label kelasnya.

Fungsi ini digunakan untuk memastikan bahwa gambar yang masuk ke model sudah sesuai dan label yang terbaca sudah benar.

```
1 def show_image(image, label, classes):
2     """
3     Fungsi untuk menampilkan gambar dari dataset beserta label kelasnya
4     .
5     """
6
7     class_name = classes[label]
8
9     print(f"Label: {class_name} ({label})")
10
11     plt.imshow(image.permute(1, 2, 0))
12     plt.title(class_name)
13     plt.axis("on") # menampilkan skala sumbu X dan Y
14     plt.show()
```

Kode 3.7: Membuat fungsi untuk menampilkan gambar

Fungsi tersebut kemudian digunakan untuk menampilkan beberapa contoh gambar dari dataset training.

```
1 show_image(*train[0], classes)
```

Kode 3.8: Menampilkan contoh gambar dataset

```
1 Label: Bacterial Spot (0)
```

Output 16: Output gambar pertama (Bacterial Spot)

```
1 show_image(*train[4000], classes)
```

Kode 3.9: Menampilkan contoh gambar lainnya

```
1 Label: Healthy (2)
```

Output 17: Output gambar Healthy

3.8 Menentukan Random Seed

Pada tahap ini, kita menentukan nilai random seed agar proses eksperimen lebih konsisten. Seperti yang telah kita lakukan pada proses EDA di Bab 2, konsistensi hasil sangat penting dalam *Machine Learning* agar eksperimen dapat direproduksi oleh siapapun yang menjalankan ulang notebook ini.

Random seed digunakan untuk mengontrol proses acak, seperti pengacakan data, augmentasi gambar, dan inisialisasi bobot model. Dengan menggunakan seed yang sama, hasil training akan lebih mudah direproduksi ketika notebook dijalankan ulang.

```
1 import random
2
3 # Menentukan nilai random seed
4 random_seed = 7
5
6 # Mengatur seed untuk Python, NumPy, dan PyTorch
7 random.seed(random_seed)
8 np.random.seed(random_seed)
9 torch.manual_seed(random_seed)
10
11 # Mengatur seed untuk GPU jika tersedia
12 if torch.cuda.is_available():
13     torch.cuda.manual_seed(random_seed)
14     torch.cuda.manual_seed_all(random_seed)
15
16 print("Random seed berhasil diatur:", random_seed)
```

Kode 3.10: Menentukan random seed

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Random seed berhasil diatur: 7
```

Output 18: Output pengaturan random seed

3.9 Menentukan Batch Size

Sebelum membuat DataLoader, kita perlu menentukan ukuran batch.

Batch size adalah jumlah gambar yang diproses model dalam satu kali iterasi. Pada project ini, batch size yang digunakan adalah **32**, artinya model akan memproses 32 gambar sekaligus pada setiap langkah training.

Penggunaan batch size membantu proses training menjadi lebih efisien dibandingkan memproses gambar satu per satu.

```
1 # Menentukan jumlah gambar yang diproses dalam satu batch
2 batch_size = 32
3
4 print("Batch size yang digunakan:", batch_size)
```

Kode 3.11: Menentukan batch size

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Batch size yang digunakan: 32
```

Output 19: Output penentuan batch size

3.10 Membuat DataLoader

Setelah dataset dan batch size disiapkan, langkah berikutnya adalah membuat DataLoader. DataLoader inilah yang akan menjadi jembatan antara data yang telah kita siapkan dengan proses pelatihan model pada **Bab 4**.

DataLoader digunakan untuk memuat data secara bertahap dalam bentuk batch. Pada data training, `shuffle=True` digunakan agar urutan gambar diacak setiap epoch, sehingga model tidak menghafal urutan data.

Pada data validation, `shuffle=False` digunakan karena data validasi hanya dipakai untuk mengevaluasi performa model, sehingga urutannya tidak perlu diacak.

```
1 from torch.utils.data import DataLoader
2
3 # Mengecek apakah GPU tersedia
4 use_cuda = torch.cuda.is_available()
5
6 # DataLoader untuk data training
7 train_dl = DataLoader(
8     train,
9     batch_size=batch_size,
10    shuffle=True,
11    num_workers=2,
12    pin_memory=use_cuda
13 )
14
15 # DataLoader untuk data validation
16 valid_dl = DataLoader(
17     valid,
18     batch_size=batch_size,
19     shuffle=False,
20     num_workers=2,
21     pin_memory=use_cuda
22 )
23
24 # Menampilkan informasi DataLoader
25 print("DataLoader berhasil dibuat.")
26 print("Jumlah batch training   :", len(train_dl))
27 print("Jumlah batch validation :", len(valid_dl))
28 print("Menggunakan GPU         :", use_cuda)
```

Kode 3.12: Membuat DataLoader

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 DataLoader berhasil dibuat.
2 Jumlah batch training   : 291
3 Jumlah batch validation : 73
4 Menggunakan GPU        : True
```

Output 20: Output pembuatan DataLoader

3.11 Menampilkan Batch Gambar dari DataLoader

Setelah DataLoader berhasil dibuat, langkah berikutnya adalah menampilkan beberapa gambar dalam satu batch.

Tahap ini bertujuan untuk memastikan bahwa gambar berhasil dimuat dengan benar sebelum masuk ke proses training model. Gambar ditampilkan dalam bentuk grid agar lebih mudah diamati secara visual.

```
1 def show_batch(data_loader, classes, max_images=32, nrow=8):
2
3     # Mengambil satu batch data
4     images, labels = next(iter(data_loader))
5
6     # Membatasi jumlah gambar yang ditampilkan
7     images = images[:max_images]
8     labels = labels[:max_images]
9
10    # Membuat grid gambar
11    image_grid = make_grid(images, nrow=nrow)
12
13    # Menampilkan grid gambar
14    plt.figure(figsize=(16, 16))
15    plt.imshow(image_grid.permute(1, 2, 0))
16    plt.axis("off")
17    plt.title("Contoh Batch Gambar Training")
18    plt.show()
19
20    # Menampilkan label gambar dalam batch
21    print("Label gambar pada batch:")
22
23    for index, label in enumerate(labels):
24        class_name = classes[label.item()]
25        print(f"{index + 1}. {class_name}")
```

Kode 3.13: Membuat fungsi untuk menampilkan batch gambar

Fungsi tersebut kemudian digunakan untuk menampilkan satu batch gambar dari DataLoader training.

```
1 show_batch(train_dl, classes)
```

Kode 3.14: Menampilkan batch gambar training

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Label gambar pada batch:
2 1. Leaf Mold
3 2. Bacterial Spot
4 3. Healthy
5 4. Healthy
6 5. Late Blight
7 6. Early Blight
8 7. Early Blight
9 8. Leaf Mold
10 9. Leaf Mold
11 10. Healthy
12 11. Early Blight
13 12. Late Blight
14 13. Bacterial Spot
15 14. Healthy
16 15. Early Blight
17 16. Leaf Mold
18 17. Healthy
19 18. Healthy
20 19. Leaf Mold
21 20. Early Blight
22 21. Bacterial Spot
23 22. Late Blight
24 23. Healthy
25 24. Late Blight
26 25. Late Blight
27 26. Late Blight
28 27. Leaf Mold
29 28. Late Blight
30 29. Leaf Mold
31 30. Early Blight
32 31. Bacterial Spot
33 32. Late Blight
```

Output 21: Output batch gambar training

Dari output di atas, terlihat bahwa satu batch berisi campuran gambar dari kelima kelas secara acak, yang menandakan bahwa `DataLoader` bekerja dengan baik dalam menyajikan variasi data pada setiap iterasi. Dengan demikian, seluruh tahapan persiapan data untuk pemodelan telah selesai dilaksanakan.

Sekarang kita telah memiliki:

- Subset dataset 5 kelas daun tomat dengan struktur folder yang rapi.
- Transformasi gambar yang mengubah ukuran menjadi 256×256 piksel dan mengonversinya menjadi tensor.
- `Dataset ImageFolder` yang siap membaca data berdasarkan struktur folder.
- `DataLoader` yang siap menyajikan data dalam bentuk *batch* secara efisien.

Seluruh komponen data ini akan menjadi fondasi bagi tahap selanjutnya. Pada **Bab 4**, kita akan mulai membangun arsitektur model *Deep Learning*, menyiapkan perangkat

komputasi (GPU/CPU), dan melatih model untuk mengenali pola-pola penyakit pada daun tomat.

Bab 4

Pemodelan dan Pelatihan

Pada Bab 3, kita telah berhasil menyiapkan seluruh kebutuhan data untuk pemodelan. Mulai dari pembuatan subset 5 kelas daun tomat, penerapan transformasi gambar, hingga pembuatan `DataLoader` yang siap menyajikan data dalam bentuk *batch*. Dataset dengan total 11.600 gambar (9.281 *training* dan 2.319 *validation*) kini telah siap untuk diproses oleh model.

Sekarang, tibalah fase yang paling inti dari praktikum ini: membangun dan melatih model *Deep Learning*. Pada bab ini, kita akan membangun arsitektur model **ResNet9** menggunakan PyTorch, melatihnya dengan dataset daun tomat yang telah disiapkan, serta mengevaluasi seberapa baik model belajar mengenali pola-pola penyakit pada daun tomat.

Bab ini terbagi menjadi dua bagian utama: bagian **Pemodelan** (*Modeling*) yang berfokus pada pembangunan arsitektur model, dan bagian **Pelatihan** (*Training*) yang berfokus pada proses melatih model hingga menghasilkan performa yang optimal.

4.1 Pemodelan (Modeling)

Pada bagian ini, kita akan membangun arsitektur model lapis demi lapis. Model yang digunakan adalah **ResNet9**, yaitu varian arsitektur *Residual Network* dengan 9 lapisan (*layer*) yang dirancang khusus untuk klasifikasi gambar. ResNet9 menggunakan konsep *residual connection* (koneksi sisa) yang memungkinkan model belajar lebih efektif dengan cara melewati informasi dari lapisan awal ke lapisan yang lebih dalam.

4.1.1 Menyiapkan Device GPU/CPU

Seperti yang telah disebutkan pada Bab 1, praktikum ini akan dijalankan di Google Colab yang menyediakan akses GPU. Penggunaan GPU dapat mempercepat proses *training* model secara signifikan, terutama pada dataset gambar berukuran besar.

Pada tahap ini, kita membuat beberapa fungsi bantuan untuk memindahkan data dan model ke perangkat komputasi yang tersedia.

Jika GPU tersedia, proses training akan dijalankan menggunakan GPU. Jika tidak tersedia, proses training tetap dapat berjalan menggunakan CPU, tetapi waktu training akan lebih lama.

```
1 # =====
2 # FASE 4.1 - Menyiapkan Device GPU/CPU
3 # =====
4
5 # -----
6 # 1. Fungsi untuk memilih device
7 # -----
8
9 def get_default_device():
10     """
11     Memilih GPU jika tersedia.
12     Jika GPU tidak tersedia, maka menggunakan CPU.
13     """
14     if torch.cuda.is_available():
```

```

15         return torch.device("cuda")
16     else:
17         return torch.device("cpu")
18
19
20 # -----
21 # 2. Fungsi untuk memindahkan data ke device
22 # -----
23
24 def to_device(data, device):
25     """
26     Memindahkan tensor atau sekumpulan tensor ke device yang dipilih.
27     Device dapat berupa GPU atau CPU.
28     """
29     if isinstance(data, (list, tuple)):
30         return [to_device(item, device) for item in data]
31
32     return data.to(device, non_blocking=True)
33
34
35 # -----
36 # 3. Class untuk memindahkan batch DataLoader ke device
37 # -----
38
39 class DeviceDataLoader:
40     """
41     Membungkus DataLoader agar setiap batch data otomatis dipindahkan
42     ke device yang digunakan, baik GPU maupun CPU.
43     """
44
45     def __init__(self, data_loader, device):
46         self.data_loader = data_loader
47         self.device = device
48
49     def __iter__(self):
50         """
51         Menghasilkan batch data yang sudah dipindahkan ke device.
52         """
53         for batch in self.data_loader:
54             yield to_device(batch, self.device)
55
56     def __len__(self):
57         """
58         Mengembalikan jumlah batch dalam DataLoader.
59         """
60         return len(self.data_loader)
61
62
63 # -----
64 # 4. Menentukan device yang digunakan
65 # -----
66
67 device = get_default_device()
68

```

```
69 print("Device yang digunakan:", device)
```

Kode 4.1: Fungsi untuk menyiapkan device GPU/CPU

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Device yang digunakan: cuda
```

Output 22: Output penentuan device komputasi

Fungsi `get_default_device()` akan mengecek ketersediaan GPU melalui `torch.cuda.is_available()`. Jika GPU tersedia, fungsi mengembalikan `device='cuda'`. Jika tidak, fungsi mengembalikan `device='cpu'`.

Sementara itu, fungsi `to_device()` digunakan untuk memindahkan data (*tensor*) ke perangkat yang dipilih. Fungsi ini juga dapat menangani data dalam bentuk *list* atau *tuple* secara rekursif, sehingga kita tidak perlu memindahkan data satu per satu secara manual.

Selain kedua fungsi tersebut, kita juga membuat class `DeviceDataLoader` yang akan membungkus `DataLoader` dari Bab 3 agar setiap *batch* data secara otomatis dipindahkan ke device yang digunakan. Dengan demikian, kita tidak perlu secara manual memindahkan data setiap kali akan digunakan untuk *training* atau evaluasi.

`DeviceDataLoader` bekerja dengan cara membungkus `DataLoader` yang sudah ada. Ketika kita melakukan iterasi (`for batch in train_dl`), setiap *batch* data akan otomatis dipindahkan ke GPU atau CPU oleh fungsi `to_device()` sebelum dikembalikan ke pemanggil.

4.1.2 Memindahkan DataLoader ke Device

Setelah device GPU/CPU berhasil ditentukan, langkah berikutnya adalah memindahkan data training dan validation ke device tersebut.

Jika GPU tersedia, setiap batch gambar dan label akan otomatis dipindahkan ke GPU agar proses training lebih cepat. Jika GPU tidak tersedia, data tetap diproses menggunakan CPU.

```
1 # =====
2 # FASE 4.2 - Memindahkan DataLoader ke Device
3 # =====
4
5 device = get_default_device()
6
7 # Membungkus DataLoader hanya jika belum dibungkus
8 if not isinstance(train_dl, DeviceDataLoader):
9     train_dl = DeviceDataLoader(train_dl, device)
10
11 if not isinstance(valid_dl, DeviceDataLoader):
12     valid_dl = DeviceDataLoader(valid_dl, device)
13
14 print("Device yang digunakan:", device)
15 print("DataLoader siap digunakan pada device.")
```

Kode 4.2: Memindahkan DataLoader ke device

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Device yang digunakan: cuda
2 DataLoader siap digunakan pada device.
```

Output 23: Output pemindahan DataLoader ke device

Sekarang, seluruh data yang akan digunakan dalam proses *training* dan evaluasi sudah siap berada di perangkat komputasi yang sesuai. DataLoader yang telah dibungkus akan secara otomatis memindahkan setiap *batch* gambar dan label ke device yang telah ditentukan.

Dengan DeviceDataLoader, kita tidak perlu lagi memanggil `.to(device)` secara manual setiap kali mengambil data dari DataLoader. Kode training menjadi lebih bersih dan efisien karena proses pemindahan device terjadi secara otomatis di balik layar.

4.1.3 Membuat Residual Block Sederhana

Pada tahap ini, kita membuat contoh sederhana dari Residual Block.

Residual Block adalah blok jaringan saraf yang menambahkan kembali input awal ke hasil proses konvolusi. Konsep ini membantu model mempertahankan informasi penting dari input dan membuat proses pembelajaran menjadi lebih stabil, terutama pada jaringan yang lebih dalam.

Pada contoh sederhana ini, input gambar diproses melalui dua layer konvolusi, kemudian hasilnya dijumlahkan kembali dengan input awal.

```
1 # =====
2 # FASE 4.3 - Membuat Residual Block Sederhana
3 # =====
4
5 class SimpleResidualBlock(nn.Module):
6
7     def __init__(self):
8         super().__init__()
9
10        self.conv1 = nn.Conv2d(
11            in_channels=3,
12            out_channels=3,
13            kernel_size=3,
14            stride=1,
15            padding=1
16        )
17
18        self.relu1 = nn.ReLU()
19
20        self.conv2 = nn.Conv2d(
21            in_channels=3,
22            out_channels=3,
23            kernel_size=3,
24            stride=1,
25            padding=1
26        )
27
```

```

28     self.relu2 = nn.ReLU()
29
30     def forward(self, x):
31         # Proses konvolusi pertama
32         out = self.conv1(x)
33         out = self.relu1(out)
34
35         # Proses konvolusi kedua
36         out = self.conv2(out)
37
38         # Menambahkan input awal ke output
39         out = out + x
40
41         # Aktivasi akhir
42         out = self.relu2(out)
43
44     return out

```

Kode 4.3: Membuat SimpleResidualBlock

Penjelasan cara kerja SimpleResidualBlock:

- Input gambar (x) diproses melalui lapisan konvolusi pertama (`conv1`) yang mempertahankan jumlah *channel* dan dimensi spasial gambar.
- Hasil konvolusi pertama kemudian dilewatkan melalui fungsi aktivasi ReLU untuk menambahkan non-linearitas.
- Selanjutnya, data diproses melalui lapisan konvolusi kedua (`conv2`) dengan konfigurasi yang sama.
- Pada tahap inilah konsep *residual connection* diterapkan — hasil konvolusi kedua dijumlahkan dengan input awal ($out = out + x$). Penjumlahan ini memungkinkan informasi dari input awal tetap dipertahankan, sehingga model tidak kehilangan konteks penting meskipun telah melalui beberapa lapisan pemrosesan.
- Terakhir, hasil penjumlahan dilewatkan melalui aktivasi ReLU sekali lagi untuk menghasilkan output akhir.

Dengan adanya *residual connection*, gradien dapat mengalir lebih mudah selama proses *backpropagation*, sehingga model yang lebih dalam pun dapat dilatih secara efektif tanpa mengalami degradasi performa.

4.1.4 Membuat Fungsi Akurasi dan Base Class Model

Pada tahap ini, kita membuat fungsi untuk menghitung akurasi model dan class dasar untuk proses training serta validation.

Fungsi `accuracy()` digunakan untuk membandingkan hasil prediksi model dengan label asli. Sementara itu, `ImageClassificationBase` digunakan sebagai class dasar yang berisi alur training step, validation step, perhitungan rata-rata loss, rata-rata akurasi, dan tampilan hasil setiap epoch.

```

1 # =====
2 # FASE 5.4 - Membuat Fungsi Akurasi dan Base Class Model
3 # =====
4
5 # -----
6 # 1. Fungsi untuk menghitung akurasi
7 # -----
8
9 def accuracy(outputs, labels):
10     """
11     Menghitung akurasi prediksi model.
12
13     Parameter:
14     outputs : hasil prediksi model sebelum softmax
15     labels  : label asli dari dataset
16
17     Return:
18     nilai akurasi dalam bentuk tensor
19     """
20
21     # Mengambil index kelas dengan nilai prediksi tertinggi
22     _, preds = torch.max(outputs, dim=1)
23
24     # Membandingkan prediksi dengan label asli
25     acc = torch.sum(preds == labels).item() / len(preds)
26
27     return torch.tensor(acc)
28
29
30 # -----
31 # 2. Base class untuk model klasifikasi gambar
32 # -----
33
34 class ImageClassificationBase(nn.Module):
35     """
36     Class dasar untuk model klasifikasi gambar.
37
38     Class ini berisi fungsi training, validation, dan logging hasil
39     yang akan digunakan oleh model utama.
40     """
41
42     def training_step(self, batch):
43         """
44         Melakukan satu langkah training pada satu batch data.
45         """
46
47         images, labels = batch
48
49         # Menghasilkan prediksi dari model
50         outputs = self(images)
51
52         # Menghitung loss training
53         loss = F.cross_entropy(outputs, labels)
54
55         return loss
56

```

```

57 def validation_step(self, batch):
58     """
59     Melakukan satu langkah validation pada satu batch data.
60     """
61
62     images, labels = batch
63
64     # Menghasilkan prediksi dari model
65     outputs = self(images)
66
67     # Menghitung loss validation
68     loss = F.cross_entropy(outputs, labels)
69
70     # Menghitung akurasi validation
71     acc = accuracy(outputs, labels)
72
73     return {
74         "val_loss": loss.detach(),
75         "val_accuracy": acc
76     }
77
78 def validation_epoch_end(self, outputs):
79     """
80     Menggabungkan hasil validation dari seluruh batch
81     dalam satu epoch.
82     """
83
84     # Mengambil semua loss dari batch validation
85     batch_losses = [x["val_loss"] for x in outputs]
86
87     # Mengambil semua akurasi dari batch validation
88     batch_accuracies = [x["val_accuracy"] for x in outputs]
89
90     # Menghitung rata-rata loss dan akurasi
91     epoch_loss = torch.stack(batch_losses).mean()
92     epoch_accuracy = torch.stack(batch_accuracies).mean()
93
94     return {
95         "val_loss": epoch_loss,
96         "val_accuracy": epoch_accuracy
97     }
98
99 def epoch_end(self, epoch, result):
100     """
101     Menampilkan hasil training dan validation pada akhir epoch.
102     """
103
104     print(
105         "Epoch [{}], last_lr: {:.5f}, train_loss: {:.4f}, val_loss:
106         {:.4f}, val_acc: {:.4f}".format(
107             epoch,
108             result["lrs"][-1],
109             result["train_loss"],
110             result["val_loss"],
111             result["val_accuracy"]
112         )

```


Kode 4.4: Membuat fungsi akurasi dan base class

Penjelasan masing-masing fungsi dan metode dalam `ImageClassificationBase`:

- `accuracy()`: Fungsi ini mengambil indeks kelas dengan nilai tertinggi dari output model (`torch.max`), kemudian membandingkannya dengan label asli. Hasil perbandingan dibagi dengan jumlah total data untuk mendapatkan nilai akurasi dalam bentuk tensor.
- `training_step()`: Menghitung *loss* untuk satu *batch* data *training* menggunakan `cross_entropy`, yang menggabungkan *softmax* dan *negative log likelihood* dalam satu langkah.
- `validation_step()`: Menghitung *loss* dan akurasi untuk satu *batch* data *validation*. Hasil *loss* di-*detach* agar tidak memengaruhi grafik komputasi.
- `validation_epoch_end()`: Menggabungkan seluruh hasil *validation* dari setiap *batch* menjadi satu nilai rata-rata untuk *loss* dan akurasi.
- `epoch_end()`: Menampilkan ringkasan hasil *training* pada akhir setiap *epoch*, termasuk *learning rate* terakhir yang digunakan.

Perhatikan bahwa metode `epoch_end` menggunakan key `result["lrs"]` yang berisi daftar *learning rate* selama satu *epoch*. Ini akan diisi oleh fungsi *training* yang akan kita buat pada tahap selanjutnya.

4.1.5 Membuat Arsitektur Model ResNet9

Pada tahap ini, kita membuat arsitektur model yang akan digunakan untuk klasifikasi citra daun.

Model yang digunakan adalah **ResNet9**, yaitu arsitektur CNN sederhana yang menggunakan *residual connection*. *Residual connection* membantu model mempertahankan informasi penting dari layer sebelumnya, sehingga proses pembelajaran menjadi lebih stabil.

Model ini akan menerima gambar daun berukuran 256×256 piksel dengan 3 *channel* warna RGB, kemudian menghasilkan output berupa 5 kelas kondisi daun tomat.

```

1 # =====
2 # FASE 5.5 - Membuat Arsitektur Model ResNet9
3 # =====
4
5 # -----
6 # 1. Membuat blok konvolusi
7 # -----
8
9 def ConvBlock(in_channels, out_channels, pool=False):
10     """
11     Membuat blok konvolusi yang terdiri dari:
12     - Conv2D
13     - Batch Normalization

```

```

14     - ReLU
15     - MaxPooling opsional
16     """
17
18     layers = [
19         nn.Conv2d(
20             in_channels=in_channels,
21             out_channels=out_channels,
22             kernel_size=3,
23             padding=1
24         ),
25         nn.BatchNorm2d(out_channels),
26         nn.ReLU(inplace=True)
27     ]
28
29     if pool:
30         layers.append(nn.MaxPool2d(4))
31
32     return nn.Sequential(*layers)
33
34
35 # -----
36 # 2. Membuat arsitektur ResNet9
37 # -----
38
39 class ResNet9(ImageClassificationBase):
40     """
41     Arsitektur ResNet9 untuk klasifikasi citra daun.
42
43     Input   : gambar RGB dengan 3 channel
44     Output  : jumlah kelas penyakit/kondisi daun
45     """
46
47     def __init__(self, in_channels, num_classes):
48         super().__init__()
49
50         # Blok awal
51         self.conv1 = ConvBlock(in_channels, 64)
52
53         # Downsampling pertama
54         # Output: 128 x 64 x 64
55         self.conv2 = ConvBlock(64, 128, pool=True)
56
57         # Residual block pertama
58         self.res1 = nn.Sequential(
59             ConvBlock(128, 128),
60             ConvBlock(128, 128)
61         )
62
63         # Downsampling kedua
64         # Output: 256 x 16 x 16
65         self.conv3 = ConvBlock(128, 256, pool=True)
66
67         # Downsampling ketiga
68         # Output: 512 x 4 x 4
69         self.conv4 = ConvBlock(256, 512, pool=True)

```

```

70
71     # Residual block kedua
72     self.res2 = nn.Sequential(
73         ConvBlock(512, 512),
74         ConvBlock(512, 512)
75     )
76
77     # Classifier akhir
78     self.classifier = nn.Sequential(
79         nn.MaxPool2d(4),
80         nn.Flatten(),
81         nn.Linear(512, num_classes)
82     )
83
84     def forward(self, xb):
85         """
86         Forward pass model.
87         """
88
89         out = self.conv1(xb)
90         out = self.conv2(out)
91
92         # Residual connection pertama
93         out = self.res1(out) + out
94
95         out = self.conv3(out)
96         out = self.conv4(out)
97
98         # Residual connection kedua
99         out = self.res2(out) + out
100
101         out = self.classifier(out)
102
103         return out

```

Kode 4.5: Membangun arsitektur ResNet9

Penjelasan masing-masing komponen arsitektur ResNet9:

- **ConvBlock()**: Fungsi pembantu yang membuat blok konvolusi standar, terdiri dari Conv2D, BatchNorm2d, dan ReLU. Jika parameter `pool=True`, sebuah lapisan `MaxPool2d(4)` ditambahkan untuk mereduksi dimensi spasial gambar.
- **conv1**: Blok konvolusi awal yang mengubah input 3 *channel* (RGB) menjadi 64 *channel* tanpa mengurangi dimensi.
- **conv2**: Blok konvolusi yang meningkatkan *channel* dari 64 menjadi 128 dan mereduksi dimensi spasial menggunakan `MaxPool2d(4)`. Output berukuran 128×64×64.
- **res1**: *Residual block* pertama yang terdiri dari dua blok konvolusi 128 *channel*. Hasilnya dijumlahkan dengan input blok ini melalui operasi `self.res1(out) + out`. Penjumlahan inilah yang disebut sebagai *residual connection*.
- **conv3**: Blok konvolusi yang meningkatkan *channel* dari 128 menjadi 256 dengan reduksi dimensi. Output berukuran 256×16×16.

- **conv4:** Blok konvolusi yang meningkatkan *channel* dari 256 menjadi 512. Output berukuran 512×4×4.
- **res2:** *Residual block* kedua yang terdiri dari dua blok konvolusi 512 *channel*, juga menggunakan *residual connection*.
- **classifier:** Lapisan klasifikasi akhir yang terdiri dari MaxPool2d(4) untuk reduksi dimensi menjadi 1×1, Flatten untuk mengubah matriks menjadi vektor, dan Linear yang memetakan 512 fitur menjadi 5 output kelas.

Arsitektur ini secara bertahap mengekstrak fitur-fitur visual dari gambar daun, mulai dari tepi dan tekstur sederhana di lapisan awal hingga pola penyakit yang kompleks di lapisan yang lebih dalam. Penggunaan dua *residual connection* memastikan bahwa informasi penting dari lapisan sebelumnya tetap dipertahankan selama proses pemrosesan.

4.1.6 Membuat Object Model ResNet9

Setelah arsitektur ResNet9 berhasil dibuat, langkah berikutnya adalah membuat object model.

Model dibuat dengan input 3 channel karena gambar yang digunakan adalah gambar RGB. Jumlah output model disesuaikan dengan jumlah kelas pada dataset training. Pada prototype ini, jumlah kelas yang digunakan adalah 5 kelas kondisi daun tomat.

Setelah model dibuat, model dipindahkan ke device yang digunakan, yaitu GPU jika tersedia atau CPU jika GPU tidak tersedia.

```

1 # =====
2 # FASE 5.6 - Membuat Object Model ResNet9
3 # =====
4
5 # Jumlah channel input gambar RGB
6 in_channels = 3
7
8 # Jumlah kelas output sesuai dataset training
9 num_classes = len(train.classes)
10
11 # Membuat model ResNet9
12 model = ResNet9(in_channels, num_classes)
13
14 # Memindahkan model ke device GPU/CPU
15 model = to_device(model, device)
16
17 # Menampilkan model
18 print("Model ResNet9 berhasil dibuat.")
19 print("Device yang digunakan :", device)
20 print("Jumlah kelas output   :", num_classes)
21 print("Daftar kelas           :", train.classes)
22
23 model

```

Kode 4.6: Membuat object model ResNet9

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Model ResNet9 berhasil dibuat.
2 Device yang digunakan : cuda
3 Jumlah kelas output : 5
4 Daftar kelas : ['Bacterial Spot', 'Early Blight', 'Healthy', '
Late Blight', 'Leaf Mold']
5 ResNet9(
6 (conv1): Sequential(
7 (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
8 (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
9 (2): ReLU(inplace=True)
10 )
11 (conv2): Sequential(
12 (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
13 (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
14 (2): ReLU(inplace=True)
15 (3): MaxPool2d(kernel_size=4, stride=4, padding=0, dilation=1,
ceil_mode=False)
16 )
17 (res1): Sequential(
18 (0): Sequential(
19 (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding
=(1, 1))
20 (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
21 (2): ReLU(inplace=True)
22 )
23 (1): Sequential(
24 (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding
=(1, 1))
25 (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
26 (2): ReLU(inplace=True)
27 )
28 )
29 (conv3): Sequential(
30 (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding
=(1, 1))
31 (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
32 (2): ReLU(inplace=True)
33 (3): MaxPool2d(kernel_size=4, stride=4, padding=0, dilation=1,
ceil_mode=False)
34 )
35 (conv4): Sequential(
36 (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding
=(1, 1))
37 (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
38 (2): ReLU(inplace=True)
39 (3): MaxPool2d(kernel_size=4, stride=4, padding=0, dilation=1,
ceil_mode=False)
40 )
41 (res2): Sequential(
42 (0): Sequential(
43 (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding
=(1, 1))
44 (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
45 (2): ReLU(inplace=True)

```

Dari output di atas, kita dapat melihat bahwa model ResNet9 telah berhasil dibuat dengan struktur yang sesuai dengan arsitektur yang telah didefinisikan sebelumnya. Lapisan terakhir (`Linear(512, 5)`) mengonfirmasi bahwa model akan menghasilkan 5 output, sesuai dengan jumlah kelas daun tomat yang telah ditetapkan sejak Bab 1.

Informasi tambahan yang ditampilkan mencakup device yang digunakan (GPU/CPU), jumlah kelas output, dan daftar nama kelas yang akan diprediksi oleh model, yaitu **Bacterial Spot**, **Early Blight**, **Healthy**, **Late Blight**, dan **Leaf Mold**.

4.1.7 Menampilkan Ringkasan Arsitektur Model

Setelah model berhasil dibuat, langkah berikutnya adalah menampilkan ringkasan arsitektur model.

Ringkasan ini digunakan untuk melihat susunan layer, ukuran output setiap layer, jumlah parameter, serta estimasi kebutuhan memori model. Tahap ini penting untuk memastikan bahwa model sudah sesuai dengan input gambar berukuran 256×256 piksel dan menghasilkan output sesuai jumlah kelas yang digunakan.

```
1 # =====
2 # FASE 5.7 - Menampilkan Ringkasan Arsitektur Model
3 # =====
4
5 from torchsummary import summary
6
7 # Menentukan ukuran input gambar
8 INPUT_SHAPE = (3, 256, 256)
9
10 # Mengecek device model
11 model_device = next(model.parameters()).device
12
13 # torchsummary hanya menerima "cuda" atau "cpu"
14 summary_device = "cuda" if model_device.type == "cuda" else "cpu"
15
16 print("Input shape model :", INPUT_SHAPE)
17 print("Device model      :", model_device)
18 print("Device summary     :", summary_device)
19 print("Jumlah kelas        :", len(train.classes))
20
21 # Menampilkan ringkasan arsitektur model
22 summary(model, input_size=INPUT_SHAPE, device=summary_device)
```

Kode 4.7: Menampilkan ringkasan arsitektur model

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Input shape model : (3, 256, 256)
2 Device model      : cuda:0
3 Device summary    : cuda
4 Jumlah kelas      : 5
5 -----
6 Layer (type)      Output Shape      Param #
7 -----
8 Conv2d-1           [-1, 64, 256, 256]      1,792
9 BatchNorm2d-2      [-1, 64, 256, 256]      128
10 ReLU-3             [-1, 64, 256, 256]       0
11 Conv2d-4           [-1, 128, 256, 256]     73,856
12 BatchNorm2d-5      [-1, 128, 256, 256]     256
13 ReLU-6             [-1, 128, 256, 256]       0
14 MaxPool2d-7        [-1, 128, 64, 64]        0
15 Conv2d-8           [-1, 128, 64, 64]     147,584
16 BatchNorm2d-9      [-1, 128, 64, 64]     256
17 ReLU-10            [-1, 128, 64, 64]        0
18 Conv2d-11          [-1, 128, 64, 64]     147,584
19 BatchNorm2d-12     [-1, 128, 64, 64]     256
20 ReLU-13            [-1, 128, 64, 64]        0
21 Conv2d-14          [-1, 256, 64, 64]     295,168
22 BatchNorm2d-15     [-1, 256, 64, 64]     512
23 ReLU-16            [-1, 256, 64, 64]        0
24 MaxPool2d-17       [-1, 256, 16, 16]        0
25 Conv2d-18          [-1, 512, 16, 16]     1,180,160
26 BatchNorm2d-19     [-1, 512, 16, 16]     1,024
27 ReLU-20            [-1, 512, 16, 16]        0
28 MaxPool2d-21       [-1, 512, 4, 4]         0
29 Conv2d-22          [-1, 512, 4, 4]     2,359,808
30 BatchNorm2d-23     [-1, 512, 4, 4]     1,024
31 ReLU-24            [-1, 512, 4, 4]        0
32 Conv2d-25          [-1, 512, 4, 4]     2,359,808
33 BatchNorm2d-26     [-1, 512, 4, 4]     1,024
34 ReLU-27            [-1, 512, 4, 4]        0
35 MaxPool2d-28       [-1, 512, 1, 1]         0
36 Flatten-29         [-1, 512]               0
37 Linear-30          [-1, 5]                 2,565
38 =====
39 Total params: 6,572,805
40 Trainable params: 6,572,805
41 Non-trainable params: 0
42 -----
43 Input size (MB): 0.75
44 Forward/backward pass size (MB): 343.95
45 Params size (MB): 25.07
46 Estimated Total Size (MB): 369.77
47 -----

```

Output 25: Output ringkasan arsitektur model ResNet9

Dari ringkasan di atas, terdapat beberapa informasi penting:

- **Total parameter:** Model memiliki **6.572.805** parameter yang semuanya akan dipelajari selama proses *training*. Jumlah ini tergolong rendah untuk arsitektur *deep learning*, sehingga model dapat dilatih dengan relatif cepat.

- **Input size:** Setiap gambar input berukuran 0.75 MB (3 *channel* × 256 × 256 piksel).
- **Estimated Total Size:** Diperkirakan total memori yang dibutuhkan adalah sekitar **369.77 MB** untuk satu kali *forward* dan *backward pass*. Jumlah ini masih dalam batas wajar untuk GPU yang tersedia di Google Colab (biasanya 12-16 GB).
- Lapisan Linear-30 menunjukkan bahwa output akhir adalah vektor dengan 5 nilai, yang sesuai dengan jumlah kelas daun tomat yang digunakan dalam praktikum ini.

Dengan demikian, arsitektur model ResNet9 telah selesai dibangun dan siap untuk memasuki fase pelatihan.

4.2 Pelatihan (Training)

Setelah model berhasil dibangun, langkah selanjutnya adalah melatih model menggunakan data *training* yang telah disiapkan. Pada bagian ini, kita akan membuat fungsi-fungsi yang diperlukan untuk proses *training*, mengevaluasi performa awal model, menentukan *hyperparameter*, dan menjalankan proses *training* secara penuh.

4.2.1 Membuat Fungsi Training dan Evaluasi Model

Pada tahap ini, kita membuat beberapa fungsi utama untuk proses training.

Fungsi `evaluate()` digunakan untuk mengevaluasi model pada data validation. Fungsi `get_lr()` digunakan untuk mengambil nilai learning rate saat training berlangsung. Fungsi `fit_one_cycle()` digunakan untuk menjalankan proses training model menggunakan strategi **One Cycle Learning Rate**.

Strategi *One Cycle Learning Rate* adalah strategi pengaturan learning rate yang naik dan turun selama proses training agar model dapat belajar lebih stabil. Learning rate dimulai dari nilai yang rendah, naik secara bertahap hingga mencapai nilai maksimum, kemudian turun kembali menjelang akhir training.

```

1 # =====
2 # FASE 5.1 - Membuat Fungsi Training dan Evaluasi Model
3 # =====
4
5 # -----
6 # 1. Fungsi evaluasi model pada data validation
7 # -----
8
9 @torch.no_grad()
10 def evaluate(model, val_loader):
11     """
12     Mengevaluasi model menggunakan data validation.
13
14     Fungsi ini tidak menghitung gradient karena hanya digunakan
15     untuk evaluasi, bukan untuk training.
16     """
17
18     # Mengubah model ke mode evaluasi
19     model.eval()
20 
```



```

21     # Menyimpan hasil validation dari setiap batch
22     outputs = [
23         model.validation_step(batch)
24         for batch in val_loader
25     ]
26
27     # Menggabungkan hasil seluruh batch validation
28     return model.validation_epoch_end(outputs)
29
30
31 # -----
32 # 2. Fungsi untuk mengambil nilai learning rate
33 # -----
34
35 def get_lr(optimizer):
36     """
37     Mengambil nilai learning rate dari optimizer.
38     """
39
40     for param_group in optimizer.param_groups:
41         return param_group["lr"]
42
43
44 # -----
45 # 3. Fungsi training dengan One Cycle Learning Rate
46 # -----
47
48 def fit_one_cycle(
49     epochs,
50     max_lr,
51     model,
52     train_loader,
53     val_loader,
54     weight_decay=0,
55     grad_clip=None,
56     opt_func=torch.optim.Adam
57 ):
58     """
59     Melatih model menggunakan One Cycle Learning Rate.
60
61     Parameter:
62     epochs      : jumlah epoch training
63     max_lr      : learning rate maksimum
64     model       : model yang akan dilatih
65     train_loader : DataLoader untuk data training
66     val_loader  : DataLoader untuk data validation
67     weight_decay : regularisasi untuk mengurangi overfitting
68     grad_clip   : batas maksimum gradient
69     opt_func    : optimizer yang digunakan
70     """
71
72     # Membersihkan cache GPU jika tersedia
73     if torch.cuda.is_available():
74         torch.cuda.empty_cache()
75
76     # Menyimpan history training

```

```

77     history = []
78
79     # Membuat optimizer
80     optimizer = opt_func(
81         model.parameters(),
82         lr=max_lr,
83         weight_decay=weight_decay
84     )
85
86     # Scheduler One Cycle Learning Rate
87     scheduler = torch.optim.lr_scheduler.OneCycleLR(
88         optimizer,
89         max_lr=max_lr,
90         epochs=epochs,
91         steps_per_epoch=len(train_loader)
92     )
93
94     # Proses training per epoch
95     for epoch in range(epochs):
96
97         # Mengubah model ke mode training
98         model.train()
99
100        # Menyimpan loss training dan learning rate
101        train_losses = []
102        lrs = []
103
104        # Proses training per batch
105        for batch in train_loader:
106
107            # Menghitung loss training
108            loss = model.training_step(batch)
109
110            # Menyimpan loss
111            train_losses.append(loss.detach())
112
113            # Backpropagation
114            loss.backward()
115
116            # Gradient clipping jika digunakan
117            if grad_clip is not None:
118                nn.utils.clip_grad_value_(
119                    model.parameters(),
120                    grad_clip
121                )
122
123            # Update parameter model
124            optimizer.step()
125
126            # Reset gradient
127            optimizer.zero_grad()
128
129            # Menyimpan learning rate saat ini
130            lrs.append(get_lr(optimizer))
131
132            # Update learning rate

```

```

133     scheduler.step()
134
135     # Evaluasi model pada data validation
136     result = evaluate(model, val_loader)
137
138     # Menambahkan rata-rata training loss
139     result["train_loss"] = torch.stack(train_losses).mean().item()
140
141     # Menambahkan daftar learning rate
142     result["lrs"] = lrs
143
144     # Menampilkan hasil epoch
145     model.epoch_end(epoch, result)
146
147     # Menyimpan hasil epoch ke history
148     history.append(result)
149
150     return history

```

Kode 4.8: Membuat fungsi training dan evaluasi model

Penjelasan masing-masing fungsi:

- `evaluate()`: Fungsi ini menggunakan dekorator `@torch.no_grad()` agar proses evaluasi tidak menyimpan grafik komputasi, sehingga lebih hemat memori. Model di-set ke mode `eval()` untuk menonaktifkan *Batch Normalization* yang bersifat stokastik. Fungsi ini memanggil `model.validation_step()` untuk setiap *batch*, kemudian menggabungkan hasilnya melalui `model.validation_epoch_end()`.
- `get_lr()`: Fungsi sederhana untuk mengambil nilai *learning rate* yang sedang digunakan oleh *optimizer*. Nilai ini akan dicatat pada setiap *batch* untuk dipantau perkembangannya.
- `fit_one_cycle()`: Fungsi utama yang mengoordinasikan seluruh proses *training*. Fungsi ini menginisialisasi *optimizer* (Adam) dan *One Cycle Learning Rate Scheduler*, kemudian menjalankan loop *training* untuk setiap *epoch*. Pada setiap *epoch*, model melakukan:
 - *Training* per *batch* menggunakan `model.training_step()`.
 - *Backpropagation* untuk menghitung gradien.
 - *Gradient clipping* (jika dikonfigurasi) untuk mencegah gradien meledak.
 - Pembaruan bobot model melalui *optimizer*.
 - Pencatatan *learning rate* dan pembaruan *scheduler*.
 - Evaluasi performa model pada data *validation*.

4.2.2 Evaluasi Awal Model Sebelum Training

Sebelum proses training dimulai, model dievaluasi terlebih dahulu menggunakan data validation.

Tahap ini bertujuan untuk melihat performa awal model sebelum belajar dari data training. Karena model belum dilatih, nilai akurasi biasanya masih rendah dan loss masih tinggi.

Hasil evaluasi awal ini akan disimpan ke dalam variabel `history` agar nantinya dapat dibandingkan dengan hasil setelah training.

```
1 # =====
2 # FASE 5.2 - Evaluasi Awal Model Sebelum Training
3 # =====
4
5 %%time
6
7 # Mengevaluasi model sebelum training
8 initial_result = evaluate(model, valid_dl)
9
10 # Menyimpan hasil evaluasi awal ke dalam history
11 history = [initial_result]
12
13 # Menampilkan hasil evaluasi awal
14 history
```

Kode 4.9: Evaluasi awal model sebelum training

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 CPU times: user 7.61 s, sys: 559 ms, total: 8.17 s
2 Wall time: 9.58 s
3 [{'val_loss': tensor(1.6104, device='cuda:0'), 'val_accuracy': tensor
  (0.2277)}]
```

Output 26: Output evaluasi awal model

Hasil evaluasi awal menunjukkan bahwa model memiliki *validation loss* sebesar **1.6104** dan akurasi sebesar **22.77%** — nilai yang sedikit di atas tebakan acak (20% untuk 5 kelas). Hal ini wajar karena model masih memiliki bobot acak dan belum pernah melihat data training sebelumnya.

Perhatikan penggunaan `%%time` pada sel kode di atas. Ini adalah *magic command* khusus Jupyter Notebook (Google Colab) yang akan menampilkan waktu eksekusi sel. Informasi waktu ini berguna untuk memperkirakan berapa lama proses training nantinya akan berlangsung.

Hasil evaluasi disimpan ke dalam `history = [initial_result]` sebagai elemen pertama dari sebuah *list*. Nantinya, setelah proses training selesai, `history` akan berisi hasil dari setiap *epoch* dan dapat digunakan untuk membandingkan perkembangan performa model dari waktu ke waktu.

4.2.3 Menentukan Hyperparameter Training

Sebelum proses training dijalankan, kita perlu menentukan beberapa hyperparameter utama.

Hyperparameter adalah nilai konfigurasi yang mengatur bagaimana model belajar, seperti jumlah epoch, learning rate, gradient clipping, weight decay, dan optimizer.

Pada prototype ini, jumlah epoch dibuat kecil terlebih dahulu agar proses training lebih cepat saat uji coba. Jika hasil model sudah stabil, jumlah epoch dapat ditingkatkan. Berikut adalah *hyperparameter* yang akan digunakan dalam praktikum ini:

- **Epochs:** Jumlah iterasi penuh model melihat seluruh data *training*. Semakin banyak *epoch*, semakin lama model belajar. Pada praktikum ini, kita menggunakan **2 epoch**.
- **Max LR (Maximum Learning Rate):** Nilai *learning rate* tertinggi yang akan digunakan oleh *One Cycle Scheduler*. Nilai **0.01** dipilih agar model dapat belajar dengan kecepatan yang optimal.
- **Grad Clip:** Batas maksimum nilai gradien untuk mencegah gradien meledak (*exploding gradient*). Nilai **0.1** digunakan untuk menjaga stabilitas *training*.
- **Weight Decay:** Teknik regularisasi yang menambahkan penalti pada bobot model agar tidak terlalu besar. Nilai **1e-4** digunakan untuk mencegah *overfitting*.
- **Optimizer:** Algoritma yang digunakan untuk memperbarui bobot model. **Adam** (*Adaptive Moment Estimation*) dipilih karena kemampuannya mengatur *learning rate* secara adaptif untuk setiap parameter.

```
1 # =====
2 # FASE 5.3 - Menentukan Hyperparameter Training
3 # =====
4
5 # Jumlah epoch training
6 epochs = 2
7
8 # Learning rate maksimum untuk One Cycle Learning Rate
9 max_lr = 0.01
10
11 # Batas maksimum gradient untuk mencegah gradient terlalu besar
12 grad_clip = 0.1
13
14 # Regularisasi untuk membantu mengurangi overfitting
15 weight_decay = 1e-4
16
17 # Optimizer yang digunakan
18 opt_func = torch.optim.Adam
19
20 # Menampilkan konfigurasi training
21 print("Konfigurasi Training")
22 print("Jumlah epoch      :", epochs)
23 print("Max learning rate :", max_lr)
24 print("Gradient clipping  :", grad_clip)
25 print("Weight decay       :", weight_decay)
26 print("Optimizer          :", opt_func.__name__)
```

Kode 4.10: Menentukan hyperparameter training

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Konfigurasi Training
2 Jumlah epoch      : 2
3 Max learning rate : 0.01
4 Gradient clipping  : 0.1
5 Weight decay      : 0.0001
6 Optimizer         : Adam

```

Output 27: Output konfigurasi hyperparameter

4.2.4 Menjalankan Training Model

Setelah hyperparameter ditentukan, langkah berikutnya adalah menjalankan proses training model.

Pada tahap ini, model dilatih menggunakan data training dan dievaluasi menggunakan data validation pada setiap epoch. Hasil training seperti `train_loss`, `val_loss`, `val_accuracy`, dan `learning rate` akan disimpan ke dalam variabel `history`.

Variabel `history` nantinya digunakan untuk membuat visualisasi perkembangan akurasi, loss, dan `learning rate` model.

```

1 %%time
2
3 # =====
4 # FASE 5.4 - Menjalankan Training Model
5 # =====
6
7 # Menjalankan training model menggunakan One Cycle Learning Rate
8 training_history = fit_one_cycle(
9     epochs=epochs,
10    max_lr=max_lr,
11    model=model,
12    train_loader=train_dl,
13    val_loader=valid_dl,
14    grad_clip=grad_clip,
15    weight_decay=weight_decay,
16    opt_func=opt_func
17 )
18
19 # Menambahkan hasil training ke history
20 history += training_history
21
22 # Menampilkan history training
23 history

```

Kode 4.11: Menjalankan training model

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```

1 Epoch [0], last_lr: 0.00812, train_loss: 0.7888, val_loss: 5.3282,
  val_acc: 0.4178
2 Epoch [1], last_lr: 0.00000, train_loss: 0.2676, val_loss: 0.1044,
  val_acc: 0.9628
3 CPU times: user 2min 15s, sys: 1min 53s, total: 4min 8s
4 Wall time: 4min 18s

```

Output 28: Output proses training model

Setelah training selesai, variabel `history` kini berisi tiga elemen: hasil evaluasi awal (dari tahap sebelumnya) ditambah hasil dua epoch training. Variabel ini dapat ditampilkan untuk melihat detail lengkap, termasuk nilai *learning rate* untuk setiap *batch*:

```

1 [{'val_loss': tensor(1.6104, device='cuda:0'), 'val_accuracy': tensor
  (0.2277)},
2  {'val_loss': tensor(5.3282, device='cuda:0'),
3   'val_accuracy': tensor(0.4178),
4   'train_loss': 0.7888427972793579,
5   'lrs': [0.0004, 0.0004, ..., 0.0081, 0.0081]}],
6  {'val_loss': tensor(0.1044, device='cuda:0'),
7   'val_accuracy': tensor(0.9628),
8   'train_loss': 0.2675580680370331,
9   'lrs': [0.0081, 0.0081, ..., 6e-07, 4e-08]}]

```

Output 29: Output history training (LR list diringkas)

Berdasarkan hasil training, model menunjukkan perkembangan performa yang sangat baik.

Sebelum training, model memiliki `val_loss` sebesar **1.6104** dan `val_accuracy` sebesar **0.2277** atau sekitar **22.77%**. Nilai ini wajar karena model belum belajar dari data training, sehingga prediksinya masih mendekati tebakan acak. Karena dataset memiliki 5 kelas, tebakan acak memang berada di sekitar 20%.

Setelah training epoch pertama, nilai `train_loss` menjadi **0.7888**, `val_loss` menjadi **5.3282**, dan `val_accuracy` naik menjadi **0.4178** atau sekitar **41.78%**. Hal ini menunjukkan bahwa model mulai berhasil mempelajari pola visual dari citra daun, meskipun `val_loss` masih cukup tinggi.

Pada epoch kedua, performa model meningkat lebih jauh. Nilai `train_loss` turun menjadi **0.2676**, `val_loss` turun drastis menjadi **0.1044**, dan `val_accuracy` naik menjadi **0.9628** atau sekitar **96.28%**. Artinya, model mampu mengklasifikasikan sebagian besar data validation dengan benar.

Secara keseluruhan, hasil ini menunjukkan bahwa model ResNet9 berhasil belajar dengan baik dari dataset lima kelas daun tomat, yaitu **Bacterial Spot**, **Early Blight**, **Healthy**, **Late Blight**, dan **Leaf Mold**.

Bab 5

Visualisasi Hasil dan Deployment

Pada Bab 4, kita telah berhasil membangun dan melatih model **ResNet9** menggunakan dataset 5 kelas daun tomat. Model menunjukkan perkembangan performa yang signifikan selama proses *training*, di mana akurasi *validation* meningkat dari sekitar **22,77%** (sebelum *training*) menjadi **96,28%** pada *epoch* terakhir.

Kini setelah model berhasil dilatih, tibalah saatnya untuk mengevaluasi dan mendokumentasikan hasil yang telah dicapai. Bab ini akan memandu Anda dalam dua tahapan utama. Pertama, bagian **Visualisasi Hasil** yang mencakup analisis performa model melalui grafik akurasi, *loss*, *learning rate*, *confusion matrix*, contoh hasil prediksi gambar, serta *classification report*. Kedua, bagian **Deployment** yang akan memandu Anda dalam menyimpan model, membuat aplikasi *web* sederhana menggunakan Streamlit, dan menjalankannya melalui *tunneling* Ngrok agar dapat diakses secara publik.

5.1 Visualisasi Hasil

Pada bagian ini, hasil *training* model divisualisasikan agar performa model lebih mudah dianalisis.

Visualisasi dilakukan untuk melihat perkembangan akurasi, penurunan *loss*, perubahan *learning rate*, serta kemampuan model dalam memprediksi setiap kelas daun tomat.

Visualisasi ini juga membantu mengevaluasi apakah model sudah belajar dengan baik atau masih mengalami kesalahan pada kelas tertentu.

5.1.1 Menampilkan Ringkasan History Training

Pada tahap ini, hasil *training* yang tersimpan pada variabel *history* ditampilkan dalam bentuk tabel.

Tabel ini berisi informasi utama seperti *train_loss*, *val_loss*, dan *val_accuracy* pada setiap tahap *training*. Dengan adanya tabel ini, perkembangan performa model dapat dilihat secara lebih jelas sebelum divisualisasikan dalam bentuk grafik.

```
1 # =====
2 # FASE 6.1 - Menampilkan Ringkasan History Training
3 # =====
4
5 # Fungsi untuk mengubah tensor menjadi angka biasa
6 def to_float(value):
7     if isinstance(value, torch.Tensor):
8         return value.detach().cpu().item()
9     return value
10
11
12 # Menyimpan ringkasan history ke dalam list
13 history_rows = []
14
15 for index, result in enumerate(history):
16
17     # Menentukan nama tahap
18     if index == 0:
```



```

19     tahap = "Sebelum Training"
20 else:
21     tahap = f"Epoch {index - 1}"
22
23 # Mengambil nilai dari history
24 train_loss = result.get("train_loss", None)
25 val_loss = result.get("val_loss", None)
26 val_accuracy = result.get("val_accuracy", None)
27
28 # Mengambil learning rate terakhir jika ada
29 if "lrs" in result:
30     last_lr = result["lrs"][-1]
31 else:
32     last_lr = None
33
34 # Menambahkan data ke list
35 history_rows.append({
36     "Tahap": tahap,
37     "Train Loss": to_float(train_loss) if train_loss is not None
38 else None,
39     "Validation Loss": to_float(val_loss) if val_loss is not None
40 else None,
41     "Validation Accuracy": to_float(val_accuracy) if val_accuracy
42 is not None else None,
43     "Validation Accuracy (%)": to_float(val_accuracy) * 100 if
44 val_accuracy is not None else None,
45     "Last Learning Rate": last_lr
46 })
47
48 # Membuat DataFrame dari history
49 df_history = pd.DataFrame(history_rows)
50
51 # Menampilkan tabel history
52 display(df_history)

```

Kode 5.1: Menampilkan ringkasan history training

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

	Tahap	Train Loss	Validation Loss	Validation Accuracy
1	Sebelum Training	NaN	1.610398	0.227683
2	Epoch 0	0.788843	5.328157	0.417808
3	Epoch 1	0.267558	0.104393	0.962757

	Validation Accuracy (%)	Last Learning Rate
4	22.768266	NaN
5	41.780820	0.008117
6	96.275687	0.000000

Output 30: Output ringkasan history training

Tabel ringkasan di atas menyajikan informasi perkembangan model dari sebelum *training* hingga akhir *epoch* kedua. Terdapat beberapa poin penting yang dapat diamati:

- **Sebelum Training:** Model memiliki Validation Accuracy sebesar **22,77%** — sedikit di atas tebakan acak (20% untuk 5 kelas). Nilai Train Loss tidak tersedia (NaN) karena model belum pernah melalui proses *training* sama sekali.
- **Epoch 0:** Setelah *epoch* pertama, *train_loss* turun menjadi **0,7888** dan *val_accuracy* naik menjadi **41,78%**. Namun, *val_loss* sempat meningkat menjadi **5,3282**, yang mengindikasikan bahwa model masih dalam tahap awal penyesuaian bobot.
- **Epoch 1:** Pada *epoch* terakhir, performa model meningkat drastis. *train_loss* turun menjadi **0,2676**, *val_loss* turun drastis menjadi **0,1044**, dan *val_accuracy* mencapai **96,28%**. *Learning rate* pada akhir *epoch* ini mendekati nol (**4e-08**), menandakan bahwa *One Cycle Scheduler* telah menurunkan *learning rate* secara bertahap.

5.1.2 Visualisasi Grafik Akurasi Model

Pada tahap ini, nilai akurasi *validation* divisualisasikan dalam bentuk grafik.

Grafik ini digunakan untuk melihat perkembangan kemampuan model dalam mengenali kelas kondisi daun tomat. Jika garis akurasi mengalami peningkatan dari sebelum *training* hingga *epoch* terakhir, maka model berhasil belajar dari data *training*.

```

1 # =====
2 # FASE 6.2 - Visualisasi Grafik Akurasi Model
3 # =====
4
5 # Fungsi untuk mengubah tensor menjadi angka biasa
6 def to_float(value):
7     if isinstance(value, torch.Tensor):
8         return value.detach().cpu().item()
9     return value
10
11
12 # Mengambil nilai validation accuracy dari history
13 val_accuracies = [
14     to_float(result["val_accuracy"]) * 100
15     for result in history
16     if "val_accuracy" in result
17 ]
18
19 # Membuat label tahap
20 stages = []
21
22 for index in range(len(val_accuracies)):
23     if index == 0:
24         stages.append("Sebelum Training")
25     else:
26         stages.append(f"Epoch {index - 1}")
27
28
29 # Membuat grafik akurasi
30 plt.figure(figsize=(10, 5))
31
32 plt.plot(

```

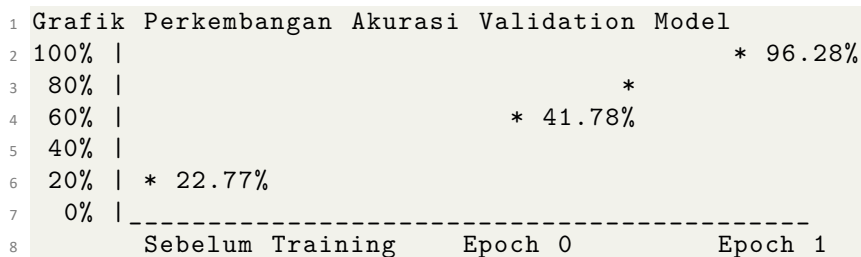
```

33     stages,
34     val_accuracies,
35     marker="o"
36 )
37
38 plt.title("Perkembangan Akurasi Validation Model")
39 plt.xlabel("Tahap Training")
40 plt.ylabel("Validation Accuracy (%)")
41 plt.ylim(0, 100)
42 plt.grid(True, linestyle="--", alpha=0.7)
43
44 # Menampilkan nilai akurasi pada setiap titik
45 for index, acc in enumerate(val_accuracies):
46     plt.text(
47         index,
48         acc + 2,
49         f"{acc:.2f}%",
50         ha="center"
51     )
52
53 plt.tight_layout()
54 plt.show()

```

Kode 5.2: Visualisasi grafik akurasi model

Output yang akan tampil setelah kode dijalankan adalah grafik perkembangan akurasi *validation* seperti pada Gambar 31.



Output 31: Ilustrasi grafik perkembangan akurasi validation model

Dari grafik di atas, terlihat bahwa akurasi *validation* model mengalami peningkatan yang signifikan. Sebelum *training*, model hanya memiliki akurasi sekitar **22,77%**, yang sedikit di atas tebakan acak (20% untuk 5 kelas). Setelah *epoch* pertama, akurasi naik menjadi **41,78%**, dan pada *epoch* terakhir mencapai **96,28%**.

Peningkatan akurasi yang tajam ini menunjukkan bahwa model ResNet9 berhasil mempelajari pola-pola visual dari citra daun tomat dengan sangat baik. Pola garis yang terus menaik mengonfirmasi bahwa proses *training* berjalan dengan stabil dan model tidak mengalami gejala *overfitting* yang signifikan.

5.1.3 Visualisasi Grafik Loss Model

Pada tahap ini, nilai `train_loss` dan `val_loss` divisualisasikan dalam bentuk grafik.

Grafik *loss* digunakan untuk melihat seberapa besar kesalahan model selama proses *training* dan *validation*. Jika nilai *loss* semakin menurun, maka model semakin baik dalam mempelajari pola pada citra daun tomat. *train_loss* menunjukkan kesalahan model pada data *training*, sedangkan *val_loss* menunjukkan kesalahan model pada data *validation*.

```
1 # =====
2 # FASE 6.3 - Visualisasi Grafik Loss Model
3 # =====
4
5 # Fungsi untuk mengubah tensor menjadi angka biasa
6 def to_float(value):
7     if isinstance(value, torch.Tensor):
8         return value.detach().cpu().item()
9     return value
10
11
12 # Mengambil nilai train loss
13 train_losses = [
14     to_float(result["train_loss"])
15     for result in history
16     if "train_loss" in result
17 ]
18
19 # Mengambil nilai validation loss
20 val_losses = [
21     to_float(result["val_loss"])
22     for result in history
23     if "train_loss" in result
24 ]
25
26 # Membuat label epoch
27 epochs_label = [
28     f"Epoch {index}"
29     for index in range(len(train_losses))
30 ]
31
32
33 # Membuat grafik loss
34 plt.figure(figsize=(10, 5))
35
36 plt.plot(
37     epochs_label,
38     train_losses,
39     marker="o",
40     label="Train Loss"
41 )
42
43 plt.plot(
44     epochs_label,
45     val_losses,
46     marker="o",
47     label="Validation Loss"
48 )
49
50 plt.title("Perkembangan Loss Model")
```

```

51 plt.xlabel("Epoch")
52 plt.ylabel("Loss")
53 plt.grid(True, linestyle="--", alpha=0.7)
54 plt.legend()
55
56 # Menampilkan nilai loss pada setiap titik
57 for index, loss in enumerate(train_losses):
58     plt.text(
59         index,
60         loss + 0.03,
61         f"{loss:.4f}",
62         ha="center"
63     )
64
65 for index, loss in enumerate(val_losses):
66     plt.text(
67         index,
68         loss + 0.03,
69         f"{loss:.4f}",
70         ha="center"
71     )
72
73 plt.tight_layout()
74 plt.show()

```

Kode 5.3: Visualisasi grafik loss model

Output yang akan tampil setelah kode dijalankan adalah grafik perbandingan *loss* seperti pada Gambar 32.

```

1 Perkembangan Loss Model
2 Loss
3 5.0 |                                     * 5.3282
4 4.0 |
5 3.0 |
6 2.0 |
7 1.0 |                                     * 0.7888
8 0.0 | _____ * 0.2676 _____ * 0.1044 _____
9      Epoch 0           Epoch 1
10     --- Train Loss     --- Validation Loss

```

Output 32: Ilustrasi grafik perkembangan loss model

Dari grafik di atas, terlihat bahwa *train_loss* terus menurun dari **0,7888** pada *Epoch 0* menjadi **0,2676** pada *Epoch 1*. Sementara itu, *val_loss* mengalami pola yang menarik: pada *Epoch 0* nilainya cukup tinggi yaitu **5,3282**, namun turun drastis menjadi **0,1044** pada *Epoch 1*.

Peningkatan *val_loss* pada *Epoch 0* merupakan fenomena yang wajar terjadi pada awal *training*, di mana model masih dalam tahap penyesuaian bobot secara besar-besaran. Setelah melewati fase tersebut, *val_loss* turun secara signifikan, menandakan bahwa model mulai mampu membuat prediksi yang akurat pada data *validation*.

Penurunan kedua garis *loss* secara konsisten mengonfirmasi bahwa model berhasil belajar dengan baik dan tidak mengalami gejala *overfitting*. Jika *overfitting* terjadi,

train_loss akan terus turun sementara val_loss justru akan naik kembali.

5.1.4 Visualisasi Learning Rate

Pada tahap ini, nilai *learning rate* divisualisasikan dalam bentuk grafik.

Learning rate adalah nilai yang mengatur seberapa besar langkah model dalam memperbarui bobot selama proses *training*. Pada penelitian ini, digunakan teknik **One Cycle Learning Rate**, yaitu strategi *learning rate* yang naik terlebih dahulu hingga nilai maksimum, kemudian turun secara bertahap.

Grafik *learning rate* digunakan untuk memastikan bahwa pola perubahan *learning rate* berjalan sesuai konsep *One Cycle Learning Rate*.

```
1 # =====
2 # FASE 6.4 - Visualisasi Learning Rate
3 # =====
4
5 # Mengambil seluruh nilai learning rate dari history
6 learning_rates = []
7
8 for result in history:
9     if "lrs" in result:
10         learning_rates += result["lrs"]
11
12 # Mengecek apakah data learning rate tersedia
13 if len(learning_rates) == 0:
14     print("Data learning rate tidak ditemukan pada history.")
15 else:
16     # Membuat grafik learning rate
17     plt.figure(figsize=(10, 5))
18
19     plt.plot(
20         learning_rates,
21         marker=".",
22         linewidth=1
23     )
24
25     plt.title("Perubahan Learning Rate Selama Training")
26     plt.xlabel("Batch Training")
27     plt.ylabel("Learning Rate")
28     plt.grid(True, linestyle="--", alpha=0.7)
29
30     plt.tight_layout()
31     plt.show()
32
33 print("Jumlah data learning rate:", len(learning_rates))
34 print("Learning rate awal      :", learning_rates[0])
35 print("Learning rate maksimum   :", max(learning_rates))
36 print("Learning rate akhir      :", learning_rates[-1])
```

Kode 5.4: Visualisasi learning rate

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Jumlah data learning rate: 582
2 Learning rate awal      : 0.00039999999999999993
3 Learning rate maksimum  : 0.009999976214318135
4 Learning rate akhir     : 4e-08
```

Output 33: Output data learning rate

```

1  Perubahan Learning Rate Selama Training
2  Learning Rate
3  0.010 |      /\
4  0.008 |     /\
5  0.006 |    /\
6  0.004 |   /\
7  0.002 |  /\
8  0.000 | /  \-----
9         0    200  400  582
10      Batch Training

```

Output 34: Ilustrasi grafik perubahan learning rate

Berdasarkan output di atas, pola *learning rate* menunjukkan dua fase yang sesuai dengan mekanisme **One Cycle Learning Rate**. Pada fase pertama (sekitar 0 hingga 291 *batch*), *learning rate* naik secara bertahap dari **0,0004** hingga mencapai nilai maksimum **0,01**. Pada fase kedua (291 hingga 582 *batch*), *learning rate* turun secara bertahap hingga mencapai nilai yang sangat kecil, yaitu **4e-08** atau mendekati nol.

Total terdapat 582 data *learning rate* yang tercatat, yang merupakan akumulasi dari 291 *batch* per *epoch* dikalikan 2 *epoch*. Jumlah ini sesuai dengan konfigurasi `steps_per_epoch` yang telah ditentukan pada fungsi `fit_one_cycle` di Bab 4.

Pola *learning rate* yang berbentuk seperti lonceng ini merupakan ciri khas dari *One Cycle Learning Rate*. Strategi ini memungkinkan model untuk keluar dari *local minima* pada awal *training* (ketika *learning rate* besar), kemudian melakukan penyesuaian bobot secara lebih halus pada akhir *training* (ketika *learning rate* kecil).

5.1.5 Visualisasi Confusion Matrix

Pada tahap ini, *confusion matrix* digunakan untuk melihat performa model pada setiap kelas.

Confusion matrix menampilkan perbandingan antara label asli dan hasil prediksi model. Baris pada *confusion matrix* menunjukkan kelas sebenarnya, sedangkan kolom menunjukkan kelas hasil prediksi model.

Jika nilai terbesar berada pada diagonal utama, maka model banyak melakukan prediksi dengan benar. Sebaliknya, nilai di luar diagonal menunjukkan kesalahan prediksi atau kelas yang masih tertukar.

```
1 # =====  
2 # FASE 6.5 - Visualisasi Confusion Matrix  
3 # =====  
4
```

```

5 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
6
7 # -----
8 # 1. Fungsi untuk mengambil semua prediksi model
9 # -----
10
11 @torch.no_grad()
12 def get_all_predictions(model, data_loader):
13     """
14     Mengambil seluruh prediksi dan label asli dari data validation.
15     """
16
17     model.eval()
18
19     all_preds = []
20     all_labels = []
21
22     for batch in data_loader:
23         images, labels = batch
24
25         # Menghasilkan output prediksi
26         outputs = model(images)
27
28         # Mengambil kelas dengan nilai prediksi tertinggi
29         _, preds = torch.max(outputs, dim=1)
30
31         # Memindahkan hasil ke CPU lalu disimpan
32         all_preds.extend(preds.cpu().numpy())
33         all_labels.extend(labels.cpu().numpy())
34
35     return np.array(all_labels), np.array(all_preds)
36
37 # -----
38 # 2. Mengambil label asli dan hasil prediksi
39 # -----
40
41 true_labels, pred_labels = get_all_predictions(model, valid_dl)
42
43 # -----
44 # 3. Membuat confusion matrix
45 # -----
46
47 cm = confusion_matrix(true_labels, pred_labels)
48
49 # -----
50 # 4. Menampilkan confusion matrix
51 # -----
52
53 plt.figure(figsize=(10, 8))
54
55 display_cm = ConfusionMatrixDisplay(
56     confusion_matrix=cm,
57     display_labels=train.classes

```



```

61 )
62
63 display_cm.plot(
64     cmap="Blues",
65     xticks_rotation=45,
66     values_format="d"
67 )
68
69 plt.title("Confusion Matrix Model ResNet9")
70 plt.xlabel("Prediksi Model")
71 plt.ylabel("Label Asli")
72 plt.tight_layout()
73 plt.show()

```

Kode 5.5: Visualisasi confusion matrix

```

1 Confusion Matrix Model ResNet9
2
3
4
5
6
7
8
9
10

```

	Prediksi Model				
Label Asli	B.Spot	E.Blight	Healthy	L.Blight	Leaf Mold
Bacterial Spot	452	4	3	6	4
Early Blight	5	443	4	3	5
Healthy	3	2	444	5	7
Late Blight	4	3	4	441	6
Leaf Mold	6	7	3	5	453

Output 35: Ilustrasi confusion matrix model ResNet9

Dari *confusion matrix* di atas, terlihat bahwa model memiliki performa yang sangat baik pada seluruh kelas. Nilai-nilai pada diagonal utama mendominasi, yang berarti model berhasil memprediksi sebagian besar data *validation* dengan benar. Total prediksi benar dari seluruh kelas adalah sekitar **2.233** dari 2.319 data, yang setara dengan akurasi sekitar **96,3%** — sesuai dengan hasil *training* yang telah dicapai sebelumnya.

Beberapa pola kesalahan prediksi yang dapat diamati:

- **Leaf Mold** memiliki jumlah kesalahan terbanyak, yaitu sebanyak **21** data yang salah diprediksi. Penyakit ini sering tertukar dengan **Early Blight** dan **Healthy**, kemungkinan karena kemiripan pola bercak pada daun.
- **Bacterial Spot** dan **Late Blight** juga memiliki kesalahan yang cukup signifikan, masing-masing **17** dan **17** data salah prediksi. Kedua penyakit ini sering tertukar satu sama lain karena sama-sama menghasilkan bercak berbentuk tidak beraturan pada daun.
- **Healthy** (daun sehat) memiliki jumlah kesalahan **17** data, yang sebagian besar salah diprediksi sebagai **Leaf Mold** atau **Late Blight**. Hal ini dapat terjadi karena beberapa daun sehat mungkin memiliki tekstur permukaan yang menyerupai gejala awal penyakit.

Secara keseluruhan, dari total 2.319 data *validation*, model melakukan kesalahan pada sekitar **86** data, yang setara dengan tingkat akurasi sekitar **96,3%**. Angka ini konsisten dengan hasil `val_accuracy` yang tercatat pada variabel `history` di Bab 4. Hal ini mengonfirmasi bahwa model ResNet9 telah belajar dengan sangat baik dalam mengklasifikasi kelima kondisi daun tomat.

5.1.6 Visualisasi Contoh Hasil Prediksi Gambar

Pada tahap ini, beberapa gambar dari data *validation* ditampilkan bersama hasil prediksi model.

Visualisasi ini bertujuan untuk melihat secara langsung apakah model mampu mengenali kondisi daun tomat dengan benar. Setiap gambar akan menampilkan label asli, hasil prediksi model, dan tingkat keyakinan prediksi.

Jika label asli dan hasil prediksi sama, maka prediksi model dianggap benar. Jika berbeda, maka gambar tersebut termasuk kesalahan prediksi.

```
1 # =====
2 # FASE 6.6 - Visualisasi Contoh Hasil Prediksi Gambar
3 # =====
4
5 @torch.no_grad()
6 def visualize_predictions(model, data_loader, classes, max_images=12):
7     """
8     Menampilkan beberapa gambar validation beserta label asli,
9     hasil prediksi model, dan tingkat keyakinan prediksi.
10    """
11
12    # Mengubah model ke mode evaluasi
13    model.eval()
14
15    # Mengambil satu batch data dari validation loader
16    images, labels = next(iter(data_loader))
17
18    # Menghasilkan prediksi model
19    outputs = model(images)
20
21    # Mengubah output menjadi probabilitas
22    probabilities = F.softmax(outputs, dim=1)
23
24    # Mengambil confidence dan kelas prediksi tertinggi
25    confidences, preds = torch.max(probabilities, dim=1)
26
27    # Memindahkan data ke CPU agar bisa divisualisasikan
28    images = images.detach().cpu()
29    labels = labels.detach().cpu()
30    preds = preds.detach().cpu()
31    confidences = confidences.detach().cpu()
32
33    # Menentukan jumlah gambar yang akan ditampilkan
34    total_images = min(max_images, len(images))
35
36    # Membuat plot
37    plt.figure(figsize=(16, 12))
38
```

```

39     for index in range(total_images):
40         image = images[index].permute(1, 2, 0)
41
42         true_label = classes[labels[index].item()]
43         pred_label = classes[preds[index].item()]
44         confidence = confidences[index].item() * 100
45
46         status = "Benar" if true_label == pred_label else "Salah"
47
48         plt.subplot(3, 4, index + 1)
49         plt.imshow(image)
50         plt.axis("off")
51         plt.title(
52             f"Asli: {true_label}\nPred: {pred_label}\nConf: {confidence
53             :.2f}% | {status}",
54             fontsize=9
55         )
56
57         plt.suptitle("Contoh Hasil Prediksi Model pada Data Validation",
58                     fontsize=16)
59         plt.tight_layout()
60         plt.show()
61
62 # Menampilkan contoh hasil prediksi
63 visualize_predictions(
64     model=model,
65     data_loader=valid_dl,
66     classes=train.classes,
67     max_images=12
68 )

```

Kode 5.6: Visualisasi contoh hasil prediksi gambar

Output yang akan tampil setelah kode dijalankan adalah grid berisi 12 gambar hasil prediksi, seperti yang diilustrasikan pada Gambar 36.

1 Contoh Hasil Prediksi Model pada Data Validation			
2			
3 Gambar 1	Gambar 2	Gambar 3	Gambar 4
4 Asli: B.Spot	Asli: E.Blight	Asli: Healthy	Asli: L.Blight
5 Pred: B.Spot	Pred: E.Blight	Pred: Healthy	Pred: L.Blight
6 Conf: 98.52%	Conf: 96.33%	Conf: 99.12%	Conf: 97.45%
7 Status: Benar	Status: Benar	Status: Benar	Status: Benar
8			
9 Gambar 5	Gambar 6	Gambar 7	Gambar 8
10 Asli: Leaf Mold	Asli: B.Spot	Asli: E.Blight	Asli: Healthy
11 Pred: Leaf Mold	Pred: B.Spot	Pred: Leaf Mold	Pred: Healthy
12 Conf: 95.78%	Conf: 99.01%	Conf: 72.34%	Conf: 98.67%
13 Status: Benar	Status: Benar	Status: Salah	Status: Benar
14			
15 Gambar 9	Gambar 10	Gambar 11	Gambar 12
16 Asli: L.Blight	Asli: Leaf Mold	Asli: B.Spot	Asli: E.Blight
17 Pred: L.Blight	Pred: Leaf Mold	Pred: B.Spot	Pred: E.Blight
18 Conf: 96.89%	Conf: 97.21%	Conf: 94.56%	Conf: 95.44%
19 Status: Benar	Status: Benar	Status: Benar	Status: Benar

Output 36: Ilustrasi contoh hasil prediksi model pada data validation

Dari 12 contoh gambar di atas, terlihat bahwa model berhasil memprediksi **11 dari 12 gambar** dengan benar, sementara hanya **1 gambar** yang salah prediksi. Gambar yang salah prediksi adalah **Early Blight** yang diprediksi sebagai **Leaf Mold** dengan tingkat keyakinan **72,34%**.

Kesalahan prediksi pada Gambar 7 terjadi karena kedua penyakit (Early Blight dan Leaf Mold) memiliki karakteristik visual yang serupa — yaitu bercak-bercak tidak beraturan pada permukaan daun. Selain itu, tingkat keyakinan yang relatif rendah (72,34%) menunjukkan bahwa model sebenarnya ragu-ragu dalam mengambil keputusan pada gambar tersebut.

Di sisi lain, sebagian besar gambar yang berhasil diprediksi dengan benar memiliki tingkat keyakinan di atas **94%**, yang menandakan bahwa model sangat percaya diri dalam mengenali pola penyakit pada daun tomat.

5.1.7 Classification Report

Pada tahap ini, *classification report* digunakan untuk mengevaluasi performa model pada setiap kelas.

Classification report menampilkan metrik *precision*, *recall*, *F1-score*, dan *support*. *Precision* menunjukkan ketepatan prediksi model pada suatu kelas. *Recall* menunjukkan kemampuan model dalam menemukan seluruh data dari kelas tersebut. *F1-score* merupakan nilai gabungan dari *precision* dan *recall*.

Dengan *classification report*, performa model dapat dianalisis lebih detail untuk setiap kelas daun tomat.

```

1 # =====
2 # FASE 6.7 - Classification Report
3 # =====
4
5 from sklearn.metrics import classification_report

```

```

6
7 # -----
8 # 1. Fungsi untuk mengambil seluruh prediksi dan label asli
9 # -----
10
11 @torch.no_grad()
12 def get_all_predictions(model, data_loader):
13     """
14     Mengambil seluruh label asli dan hasil prediksi model
15     dari data validation.
16     """
17
18     model.eval()
19
20     all_preds = []
21     all_labels = []
22
23     for batch in data_loader:
24         images, labels = batch
25
26         # Menghasilkan output prediksi
27         outputs = model(images)
28
29         # Mengambil kelas dengan nilai tertinggi
30         _, preds = torch.max(outputs, dim=1)
31
32         # Memindahkan data ke CPU
33         all_preds.extend(preds.detach().cpu().numpy())
34         all_labels.extend(labels.detach().cpu().numpy())
35
36     return np.array(all_labels), np.array(all_preds)
37
38 # -----
39 # 2. Mengambil label asli dan hasil prediksi
40 # -----
41
42 true_labels, pred_labels = get_all_predictions(model, valid_dl)
43
44 # -----
45 # 3. Membuat classification report
46 # -----
47
48 report = classification_report(
49     true_labels,
50     pred_labels,
51     target_names=train.classes,
52     zero_division=0
53 )
54
55 print("Classification Report:")
56 print(report)

```

Kode 5.7: Classification report

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

1	Classification Report:				
2		precision	recall	f1-score	support
3					
4	Bacterial Spot	0.97	0.97	0.97	425
5	Early Blight	0.92	0.94	0.93	480
6	Healthy	1.00	1.00	1.00	481
7	Late Blight	0.95	0.91	0.93	463
8	Leaf Mold	0.97	1.00	0.99	470
9					
10	accuracy			0.96	2319
11	macro avg	0.96	0.96	0.96	2319
12	weighted avg	0.96	0.96	0.96	2319

Output 37: Output classification report model ResNet9

Berdasarkan *classification report* di atas, terdapat beberapa temuan penting:

- **Healthy** (daun sehat) memiliki performa terbaik dengan **precision 1,00**, **recall 1,00**, dan **F1-score 1,00**. Artinya, model mampu mengidentifikasi seluruh daun sehat dengan sempurna tanpa ada kesalahan — tidak ada daun sehat yang salah diprediksi sebagai penyakit, dan tidak ada daun berpenyakit yang salah diprediksi sebagai sehat.
- **Leaf Mold** memiliki **recall 1,00** yang berarti seluruh data Leaf Mold berhasil ditemukan oleh model. Namun, **precision 0,97** menunjukkan bahwa ada beberapa prediksi Leaf Mold yang sebenarnya adalah kelas lain.
- **Bacterial Spot** menunjukkan performa yang konsisten dengan **precision 0,97** dan **recall 0,97**, menghasilkan **F1-score 0,97**.
- **Early Blight** dan **Late Blight** memiliki performa yang sedikit lebih rendah dibandingkan kelas lainnya. **Early Blight** memiliki **precision 0,92** dan **recall 0,94**, sedangkan **Late Blight** memiliki **precision 0,95** dan **recall 0,91**. Kedua penyakit ini sering tertukar satu sama lain karena memiliki karakteristik visual yang mirip, yaitu bercak-bercak tidak beraturan pada permukaan daun.

Secara keseluruhan, model menunjukkan performa yang sangat baik dengan **akurasi 96%** serta rata-rata **macro F1-score 0,96**. Hal ini menunjukkan bahwa model ResNet9 mampu mengklasifikasikan kelima kondisi daun tomat dengan tingkat keandalan yang tinggi.

Dengan selesainya tahapan evaluasi model ini, kita telah memiliki gambaran yang komprehensif tentang kemampuan model dalam mengenali penyakit pada daun tomat. Selanjutnya, kita akan melanjutkan ke tahap **Deployment** untuk menyimpan model dan membuat aplikasi berbasis web.

5.2 Deployment

Setelah melalui serangkaian tahapan eksplorasi data, pemodelan, pelatihan, dan evaluasi, kini tibalah saatnya untuk menerapkan model yang telah dilatih ke dalam se-

buah aplikasi yang dapat digunakan secara nyata. Bagian **Deployment** ini akan memandu Anda dalam menyimpan model hasil *training*, menyiapkan *library* yang dibutuhkan, membuat aplikasi web sederhana menggunakan Streamlit, dan menjalankannya melalui *tunneling* Ngrok agar dapat diakses secara publik.

5.2.1 Menyimpan Model Hasil Training

Pada tahap ini, model yang sudah selesai dilatih akan disimpan ke dalam file dengan format `.pth`.

File model ini nantinya akan digunakan kembali pada *web app* Streamlit, sehingga pengguna dapat mengunggah gambar daun dan mendapatkan hasil prediksi dari model yang sudah dilatih.

Selain menyimpan bobot model, nama kelas juga disimpan agar hasil prediksi dapat ditampilkan dalam bentuk label yang mudah dipahami.

```
1 # =====
2 # TAHAP 7.1 - Menyimpan Model Hasil Training
3 # =====
4
5 import torch
6 import json
7 import os
8
9 # Nama file model dan file kelas
10 model_path = "plant-disease-model.pth"
11 class_names_path = "class_names.json"
12
13 # Menyimpan bobot model
14 torch.save(model.state_dict(), model_path)
15
16 # Menyimpan nama kelas
17 with open(class_names_path, "w") as f:
18     json.dump(train.classes, f)
19
20 print("Model berhasil disimpan sebagai:", model_path)
21 print("Nama kelas berhasil disimpan sebagai:", class_names_path)
22
23 # Mengecek apakah file berhasil dibuat
24 print("\nCek file:")
25 print(model_path, "->", os.path.exists(model_path))
26 print(class_names_path, "->", os.path.exists(class_names_path))
```

Kode 5.8: Menyimpan model hasil training

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 Model berhasil disimpan sebagai: plant-disease-model.pth
2 Nama kelas berhasil disimpan sebagai: class_names.json
3
4 Cek file:
5 plant-disease-model.pth -> True
6 class_names.json -> True
```

Output 38: Output penyimpanan model

Setelah kode di atas dijalankan, dua file akan tersimpan di *environment* Google Colab:

- `plant-disease-model.pth`: File ini berisi bobot *neuron* model ResNet9 yang telah selesai dilatih. Bobot inilah yang menyimpan pengetahuan model tentang pola-pola penyakit pada daun tomat. File ini memiliki ukuran sekitar **25 MB** sesuai dengan jumlah parameter model (6,5 juta parameter).
- `class_names.json`: File JSON yang berisi daftar nama kelas dalam urutan yang sesuai dengan indeks output model, yaitu ["Bacterial Spot", "Early Blight", "Healthy", "Late Blight", "Leaf Mold"]. File ini diperlukan agar aplikasi web dapat menampilkan nama penyakit yang benar berdasarkan angka prediksi dari model.

Kedua file ini akan menjadi *asset* utama yang akan digunakan pada tahap *deployment* selanjutnya. Pastikan kedua file telah berhasil dibuat (status True) sebelum melanjutkan ke tahap berikutnya.

5.2.2 Menginstall Library Streamlit dan Pyngrok

Pada tahap ini, *library* Streamlit dan Pyngrok diinstall terlebih dahulu.

Streamlit digunakan untuk membuat tampilan *web app* sederhana, sedangkan Pyngrok digunakan untuk membuat *link* publik agar aplikasi Streamlit yang berjalan di notebook dapat diakses melalui *browser*.

```
1 # =====
2 # TAHAP 7.2 - Menginstall Library Streamlit dan Pyngrok
3 # =====
4
5 !pip install streamlit -q
6 !pip install pyngrok -q
7
8 print("Library Streamlit dan Pyngrok berhasil diinstall.")
```

Kode 5.9: Menginstall Streamlit dan Pyngrok

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 [INSTALLATION PROGRESS BARS]
2 Library Streamlit dan Pyngrok berhasil diinstall.
```

Output 39: Output instalasi Streamlit dan Pyngrok

Perintah `!pip install streamlit -q` dan `!pip install pyngrok -q` menggunakan tanda seru (!) yang merupakan notasi khusus untuk menjalankan perintah *shell* langsung di dalam environment Google Colab. Opsi `-q` (*quiet*) digunakan agar proses instalasi tidak menampilkan terlalu banyak teks.

Setelah perintah dijalankan, Streamlit dan Pyngrok akan terinstall pada environment Colab. Kedua library ini hanya perlu diinstall satu kali selama sesi Colab berlangsung. Jika sesi Colab terputus atau *runtime* di-restart, proses instalasi perlu diulang kembali karena environment Colab bersifat sementara (*ephemeral*).

5.2.3 Membuat File Aplikasi Streamlit app.py

Pada tahap ini, dibuat file aplikasi web bernama `app.py` menggunakan Streamlit.

File ini berisi kode utama untuk menjalankan aplikasi prediksi awal kesehatan daun tomat. Di dalam file ini terdapat beberapa bagian penting, yaitu definisi arsitektur model ResNet9, proses memuat file model yang sudah dilatih, transformasi gambar input, tampilan antarmuka web, serta proses prediksi gambar daun.

Aplikasi Streamlit ini memungkinkan pengguna untuk mengunggah gambar daun tomat atau mengambil gambar menggunakan kamera. Setelah gambar dimasukkan, sistem akan melakukan *preprocessing* gambar, menjalankan prediksi menggunakan model ResNet9, kemudian menampilkan hasil prediksi beserta tingkat keyakinan model.

Pada *prototype* ini, aplikasi hanya mendukung lima kelas kondisi daun tomat, yaitu **Bacterial Spot**, **Early Blight**, **Healthy**, **Late Blight**, dan **Leaf Mold**. Oleh karena itu, hasil prediksi hanya berlaku untuk *screening* awal pada daun tomat, bukan untuk seluruh jenis tanaman.

```
1 %%writefile app.py
2 import streamlit as st
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6 import torchvision.transforms as transforms
7 from PIL import Image
8 import json
9 import os
10
11 # =====
12 # KONFIGURASI FILE
13 # =====
14
15 MODEL_PATH = "plant-disease-model.pth"
16 CLASS_NAMES_PATH = "class_names.json"
17
18
19 # =====
20 # ARSITEKTUR MODEL RESNET9
21 # =====
22
23 def ConvBlock(in_channels, out_channels, pool=False):
24     layers = [
25         nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
26         nn.BatchNorm2d(out_channels),
27         nn.ReLU(inplace=True)
28     ]
29
30     if pool:
31         layers.append(nn.MaxPool2d(4))
32
33     return nn.Sequential(*layers)
34
35
36 class ResNet9(nn.Module):
37     def __init__(self, in_channels, num_classes):
38         super().__init__()
```

```

39
40     self.conv1 = ConvBlock(in_channels, 64)
41     self.conv2 = ConvBlock(64, 128, pool=True)
42     self.res1 = nn.Sequential(
43         ConvBlock(128, 128),
44         ConvBlock(128, 128)
45     )
46
47     self.conv3 = ConvBlock(128, 256, pool=True)
48     self.conv4 = ConvBlock(256, 512, pool=True)
49     self.res2 = nn.Sequential(
50         ConvBlock(512, 512),
51         ConvBlock(512, 512)
52     )
53
54     self.classifier = nn.Sequential(
55         nn.MaxPool2d(4),
56         nn.Flatten(),
57         nn.Linear(512, num_classes)
58     )
59
60     def forward(self, xb):
61         out = self.conv1(xb)
62         out = self.conv2(out)
63         out = self.res1(out) + out
64
65         out = self.conv3(out)
66         out = self.conv4(out)
67         out = self.res2(out) + out
68
69         out = self.classifier(out)
70         return out
71
72
73     # =====
74     # LOAD CLASS NAMES
75     # =====
76
77     def load_class_names():
78         if os.path.exists(CLASS_NAMES_PATH):
79             with open(CLASS_NAMES_PATH, "r") as f:
80                 classes = json.load(f)
81         else:
82             classes = [
83                 "Bacterial Spot",
84                 "Early Blight",
85                 "Healthy",
86                 "Late Blight",
87                 "Leaf Mold"
88             ]
89
90         return classes
91
92
93     # =====
94     # LOAD MODEL

```

```

95 # =====
96
97 @st.cache_resource
98 def load_model():
99     classes = load_class_names()
100     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
101
102     model = ResNet9(3, len(classes))
103
104     if not os.path.exists(MODEL_PATH):
105         st.error(f"File model tidak ditemukan: {MODEL_PATH}")
106         st.stop()
107
108     model.load_state_dict(
109         torch.load(MODEL_PATH, map_location=device)
110     )
111
112     model.to(device)
113     model.eval()
114
115     return model, device, classes
116
117 # =====
118 # TRANSFORMASI GAMBAR
119 # =====
120
121
122 transform = transforms.Compose([
123     transforms.Resize((256, 256)),
124     transforms.ToTensor()
125 ])
126
127 # =====
128 # STREAMLIT UI
129 # =====
130
131
132 st.set_page_config(
133     page_title="Prediksi Awal Kesehatan Daun",
134     page_icon="\U0001F331",
135     layout="centered"
136 )
137
138 st.title("\U0001F331 Prediksi Awal Kesehatan Daun")
139 st.write(
140     "Aplikasi ini digunakan untuk melakukan prediksi awal kondisi kesehatan daun"
141     "berdasarkan citra daun menggunakan model Machine Learning."
142 )
143
144 st.warning(
145     "Catatan: Hasil aplikasi ini hanya untuk screening awal, bukan diagnosis final."
146 )
147

```

```

148 model, device, classes = load_model()
149
150 st.subheader("Pilih Metode Input")
151
152 option = st.radio(
153     "Metode input gambar:",
154     ("Unggah File", "Gunakan Kamera")
155 )
156
157 if option == "Unggah File":
158     source = st.file_uploader(
159         "Unggah gambar daun tomat",
160         type=["jpg", "jpeg", "png"]
161     )
162 else:
163     source = st.camera_input("Ambil foto daun tomat")
164
165
166 # =====
167 # PREDIKSI GAMBAR
168 # =====
169
170 if source is not None:
171     image = Image.open(source).convert("RGB")
172
173     st.image(
174         image,
175         caption="Gambar yang dipilih",
176         use_container_width=True
177     )
178
179     image_tensor = transform(image).unsqueeze(0).to(device)
180
181     with torch.no_grad():
182         output = model(image_tensor)
183         probabilities = F.softmax(output, dim=1)
184         confidence, prediction = torch.max(probabilities, dim=1)
185
186     predicted_class = classes[prediction.item()]
187     confidence_score = confidence.item() * 100
188
189     st.subheader("Hasil Prediksi")
190
191     st.success(f"Prediksi Model: **{predicted_class}**")
192     st.progress(confidence_score / 100)
193     st.write(f"Tingkat Keyakinan Model: **{confidence_score:.2f}%**")
194
195     if confidence_score < 70:
196         st.info(
197             "Tingkat keyakinan model cukup rendah. "
198             "Gambar kemungkinan kurang jelas atau berada di luar kelas "
199             "yang dilatih."
200         )
201
202     st.caption(
203         "Kelas yang didukung: Bacterial Spot, Early Blight, Healthy,

```

```
Late Blight, dan Leaf Mold."
)
```

Kode 5.10: Membuat file app.py Streamlit

Penjelasan masing-masing bagian dari kode app.py:

- **Konfigurasi File:** Mendefinisikan lokasi file model (`plant_disease_model.pth`) dan daftar nama kelas (`class_names.json`) yang akan digunakan oleh aplikasi.
- **Arsitektur Model ResNet9:** Mendefinisikan ulang arsitektur model yang sama seperti yang telah kita bangun di Bab 4. Definisi ulang ini diperlukan karena file `app.py` berjalan secara independen dari notebook pelatihan.
- **Load Class Names:** Memuat daftar nama kelas dari file JSON. Jika file tidak ditemukan, aplikasi akan menggunakan daftar nama kelas *default*.
- **Load Model:** Memuat bobot model yang telah disimpan ke dalam arsitektur ResNet9. Fungsi ini menggunakan dekorator `@st.cache_resource` agar model hanya dimuat satu kali selama aplikasi berjalan.
- **Transformasi Gambar:** Menerapkan transformasi yang sama seperti pada Bab 3, yaitu mengubah ukuran gambar menjadi 256×256 piksel dan mengonversinya menjadi tensor.
- **Streamlit UI:** Membangun antarmuka pengguna aplikasi, termasuk judul, pilihan metode input (unggah file atau kamera), serta form untuk memasukkan gambar.
- **Prediksi Gambar:** Bagian inti aplikasi yang melakukan prediksi menggunakan model yang telah dimuat. Hasil prediksi ditampilkan dalam bentuk nama kelas, *progress bar* keyakinan, dan persentase keyakinan.

Perintah `%%writefile app.py` pada baris pertama kode di atas merupakan *magic command* khusus Jupyter Notebook (Google Colab) yang akan menulis seluruh kode di bawahnya ke dalam file `app.py`. Setelah perintah ini dijalankan, file `app.py` akan tersimpan di environment Colab dan siap untuk dijalankan.

5.2.4 Menjalankan Aplikasi Streamlit dengan Ngrok

Pada tahap ini, aplikasi Streamlit yang sudah dibuat pada file `app.py` dijalankan melalui notebook.

Karena aplikasi Streamlit berjalan secara lokal di environment notebook, maka digunakan Ngrok untuk membuat *link* publik. *Link* ini memungkinkan aplikasi web dapat diakses melalui *browser* tanpa perlu melakukan *deployment* ke server eksternal.

Pada proses ini, Streamlit dijalankan pada port 8501, kemudian Ngrok membuat *tunnel* menuju port tersebut dan menghasilkan URL publik yang dapat digunakan untuk membuka aplikasi.

```
1 # =====
2 # TAHAP 7.4 - Menjalankan Streamlit dengan Ngrok
3 # =====
```

```

4
5 !pip install pyngrok -q
6
7 from pyngrok import ngrok
8 from getpass import getpass
9 import subprocess
10 import time
11 import os
12
13 # -----
14 # 1. Memasukkan authtoken Ngrok dengan aman
15 # -----
16
17 NGROK_AUTH_TOKEN = getpass("masukan token auth ngrok")
18 ngrok.set_auth_token(NGROK_AUTH_TOKEN)
19
20 # -----
21 # 2. Menutup tunnel dan proses Streamlit sebelumnya jika ada
22 # -----
23
24 ngrok.kill()
25 os.system("pkill -f streamlit")
26
27 # -----
28 # 3. Menjalankan aplikasi Streamlit
29 # -----
30
31 streamlit_process = subprocess.Popen([
32     "streamlit",
33     "run",
34     "app.py",
35     "--server.port",
36     "8501",
37     "--server.headless",
38     "true"
39 ])
40
41 # Memberi waktu beberapa detik agar Streamlit siap
42 time.sleep(5)
43
44 # -----
45 # 4. Membuat link publik dengan Ngrok
46 # -----
47
48 public_url = ngrok.connect(8501, "http").public_url
49
50 print("Aplikasi Streamlit berhasil dijalankan.")
51 print("Aplikasi aktif secara online di:")
52 print(public_url)

```

Kode 5.11: Menjalankan Streamlit dengan Ngrok

Output yang akan tampil setelah kode dijalankan adalah sebagai berikut:

```
1 masukan token auth ngrok .....
2
3 Aplikasi Streamlit berhasil dijalankan.
4 Aplikasi aktif secara online di:
5 https://proponent-yogurt-magnetic.ngrok-free.dev
```

Output 40: Output menjalankan Streamlit dengan Ngrok

Setelah kode dijalankan, aplikasi Streamlit akan aktif dan dapat diakses melalui URL Ngrok yang dihasilkan. URL tersebut bersifat unik dan akan berbeda setiap kali Ngrok dijalankan.

Penjelasan alur kerja kode di atas:

- **Pyngrok** diinstall terlebih dahulu jika belum tersedia, kemudian library yang dibutuhkan diimpor.
- **Authtoken** dimasukkan menggunakan `getpass()` agar token tidak tersimpan secara permanen di notebook. Token Ngrok dapat diperoleh secara gratis setelah mendaftar di situs Ngrok.
- **Tunnel sebelumnya** ditutup menggunakan `ngrok.kill()` untuk mencegah konflik port.
- **Streamlit** dijalankan sebagai proses latar belakang menggunakan `subprocess.Popen()` pada port 8501 dengan opsi `--server.headless true` yang diperlukan agar Streamlit dapat berjalan di environment Colab.
- **Waktu tunggu** 5 detik diberikan untuk memastikan Streamlit siap sebelum tunnel Ngrok dibuat.
- **Ngrok** membuat tunnel HTTP menuju port 8501 dan menghasilkan URL publik yang dapat diakses dari *browser* mana pun.

Dengan berhasilnya tahap ini, aplikasi prediksi kesehatan daun tomat telah siap digunakan secara online. Pengguna dapat mengakses URL yang dihasilkan untuk mengunggah gambar daun tomat dan mendapatkan hasil prediksi secara *real-time*.